

Reconnaissance and Information Gathering

Executive Summary

This lab focused on advanced reconnaissance and information gathering using Kali Linux tools such as ping, nslookup, dig, whois, theHarvester, and Sublist3r. The objective was to collect and analyze publicly available information about target domains (e.g., google.com, github.com) to understand their network infrastructure and exposure level.

The results revealed highly structured and globally distributed systems with strong DNS configuration, multiple IP ranges, and extensive subdomain usage, indicating large-scale enterprise security architecture.

Tools Used

- ping
- nslookup
- dig
- whois
- theHarvester
- Sublist3r
- Bash scripting (automation)

Part 1: Fundamental Reconnaissance Techniques

Exercise 1.1: Advanced Target Profiling with `ping`

Step 1: Basic Ping Analysis

bash:

ping -c 4 [google.com](https://www.google.com)

```
(kali㉿kali)-[~/INT302_Lab1]
$ ping -c 4 google.com
PING google.com (172.217.22.46) 56(84) bytes of data.
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=170 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=152 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=151 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=224 ms

— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 7180ms
rtt min/avg/max/mdev = 151.061/174.008/223.572/29.566 ms

(kali㉿kali)-[~/INT302_Lab1]
$
```

The target host responded successfully with 0% packet loss, confirming stable connectivity to Google's server. Average latency was 174 ms, indicating moderate network delay between the local machine and the destination.

Step 2: Advanced Ping Techniques

bash:

ping -D -c 5 google.com

ping -f -c 100 target-domain.com

ping -s 1000 -c 4 google.com

ping -R -c 4 [google.com](https://www.google.com)

```
kali@kali: ~/INT302_Lab1
$ ping -D -c 5 google.com
PING google.com (172.217.22.46) 56(84) bytes of data.
[1779350561.233188] 64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=288 ms
[1779350562.263692] 64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=323 ms
[1779350563.126052] 64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=185 ms
[1779350564.330298] 64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=387 ms
[1779350565.131427] 64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=5 ttl=110 time=188 ms

--- google.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4015ms
rtt min/avg/max/mdev = 184.756/274.267/387.331/78.555 ms

(kali@kali)~/INT302_Lab1
$ sudo ping -f -c 100 google.com
[sudo] password for kali:
PING google.com (172.217.22.46) 56(84) bytes of data.
.....
--- google.com ping statistics ---
100 packets transmitted, 92 received, 8% packet loss, time 1638ms
rtt min/avg/max/mdev = 129.446/172.104/231.997/20.855 ms, pipe 14, ipg/ewma 16.545/171.726 ms

(kali@kali)~/INT302_Lab1
$ ping -s 1000 -c 4 google.com
PING google.com (172.217.22.46) 1000(1028) bytes of data.
1008 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=195 ms
1008 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=224 ms
1008 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=180 ms
1008 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=175 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3011ms
rtt min/avg/max/mdev = 174.847/193.507/224.392/19.373 ms

(kali@kali)~/INT302_Lab1
$ ping -R -c 4 google.com
PING google.com (172.217.22.46) 56(124) bytes of data.
From 10.226.228.4 icmp_seq=1 Time to live exceeded
From 10.226.228.4 icmp_seq=2 Time to live exceeded
From 10.226.228.4 icmp_seq=3 Time to live exceeded
From 10.226.228.4 icmp_seq=4 Time to live exceeded

--- google.com ping statistics ---
4 packets transmitted, 0 received, +4 errors, 100% packet loss, time 3009ms
```

ping -D -c 5 google.com

The target host responded successfully with 0% packet loss, confirming stable connectivity to Google's server. Average latency was 274 ms, indicating moderate network delay between the local machine and the destination.

sudo ping -f -c 100 google.com

Flood ping sent 100 rapid ICMP packets to stress test connectivity. 8% packet

loss occurred under high traffic load. This suggests the network or target rate limited some packets during intensive transmission.

ping -s 1000 -c 4 google.com

Large ICMP packets (1000 bytes) were transmitted successfully with 0% packet loss, showing the network can handle larger packet sizes reliably. Average response time remained stable at 193 ms.

ping -R -c 4 google.com

Record-route ping failed with Time to live exceeded errors and 100% packet loss, indicating intermediate routers blocked or failed to process the route-recording option. This is common due to modern network security restrictions.

Step 3: Create a Target Analysis Script

bash

```
nano basic_recon.sh
'''
```

bash

```
#!/bin/bash
# Basic Reconnaissance Script
# Usage: ./basic_recon.sh <domain>
```

```
DOMAIN="$1"
```

```
LOG_FILE="recon_${DOMAIN}_${date +%Y%m%d_%H%M%S}.txt"
```

```
echo "🎯 Starting Basic Reconnaissance for: $DOMAIN" | tee "$LOG_FILE"
```

```
echo "===== " | tee -a "$LOG_FILE"
```

```
# Function to perform ping analysis
```

```
ping_analysis() {
```

```
    echo "🔍 Ping Analysis for $DOMAIN" | tee -a "$LOG_FILE"
```

```
    echo "-----" | tee -a "$LOG_FILE"
```

```
    if ping -c 4 "$DOMAIN" > /dev/null 2>&1; then
```

```
echo "✅ Target is reachable" | tee -a "$LOG_FILE"  
ping -c 4 "$DOMAIN" | tee -a "$LOG_FILE"
```

```
# Extract IP address from ping
```

```
IP_ADDRESS=$(ping -c 1 "$DOMAIN" | head -1 | cut -d'(' -f2 | cut -d')' -f1)
```

```
echo "📡 IP Address: $IP_ADDRESS" | tee -a "$LOG_FILE"
```

```
else
```

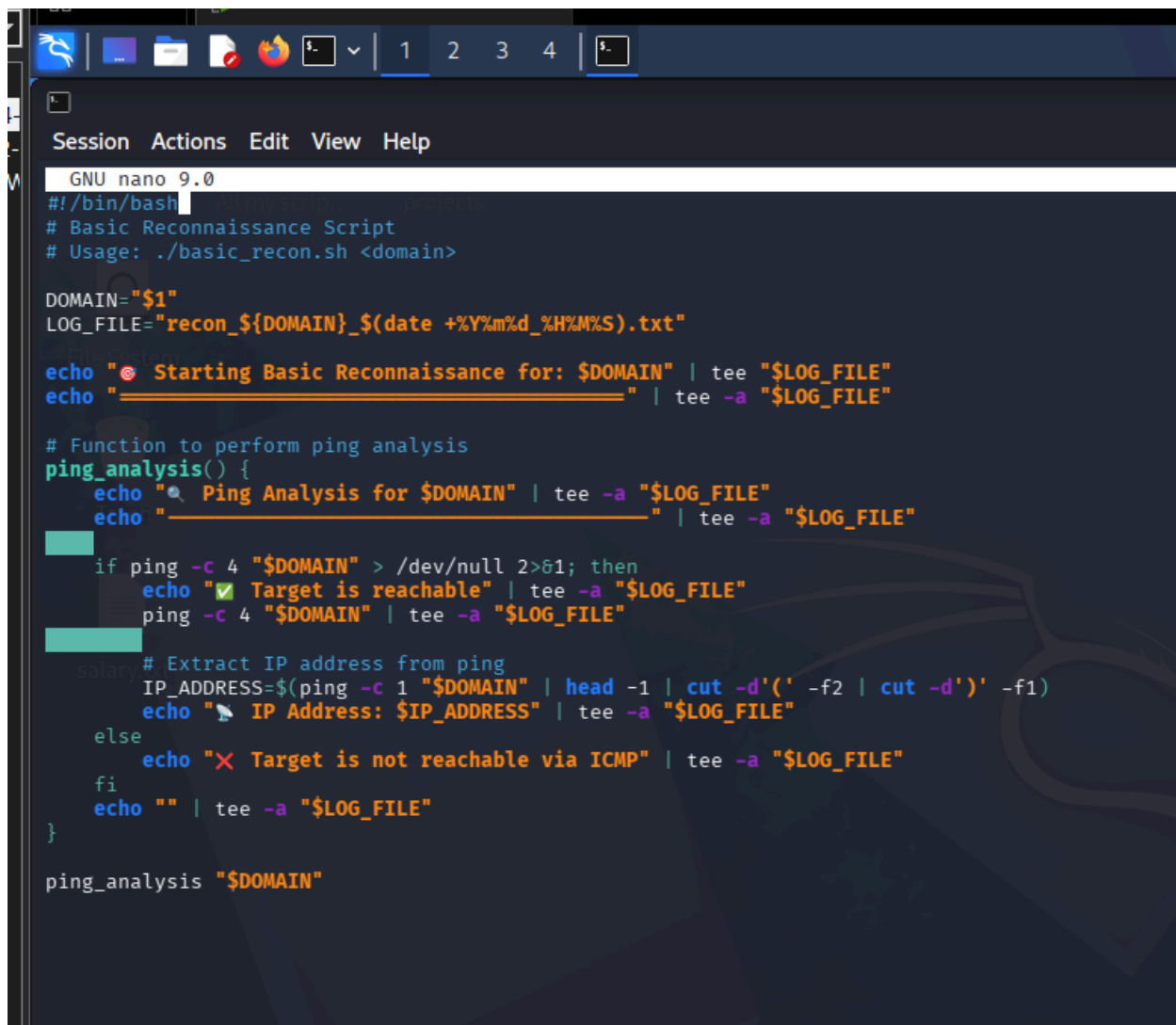
```
echo "❌ Target is not reachable via ICMP" | tee -a "$LOG_FILE"
```

```
fi
```

```
echo "" | tee -a "$LOG_FILE"
```

```
}
```

```
ping_analysis "$DOMAIN"
```



```
GNU nano 9.0
#!/bin/bash
# Basic Reconnaissance Script
# Usage: ./basic_recon.sh <domain>

DOMAIN="$1"
LOG_FILE="recon_${DOMAIN}_${date +%Y%m%d_%H%M%S}.txt"

echo "🔍 Starting Basic Reconnaissance for: $DOMAIN" | tee "$LOG_FILE"
echo "===== " | tee -a "$LOG_FILE"

# Function to perform ping analysis
ping_analysis() {
    echo "🔍 Ping Analysis for $DOMAIN" | tee -a "$LOG_FILE"
    echo "===== " | tee -a "$LOG_FILE"

    if ping -c 4 "$DOMAIN" > /dev/null 2>&1; then
        echo "✅ Target is reachable" | tee -a "$LOG_FILE"
        ping -c 4 "$DOMAIN" | tee -a "$LOG_FILE"

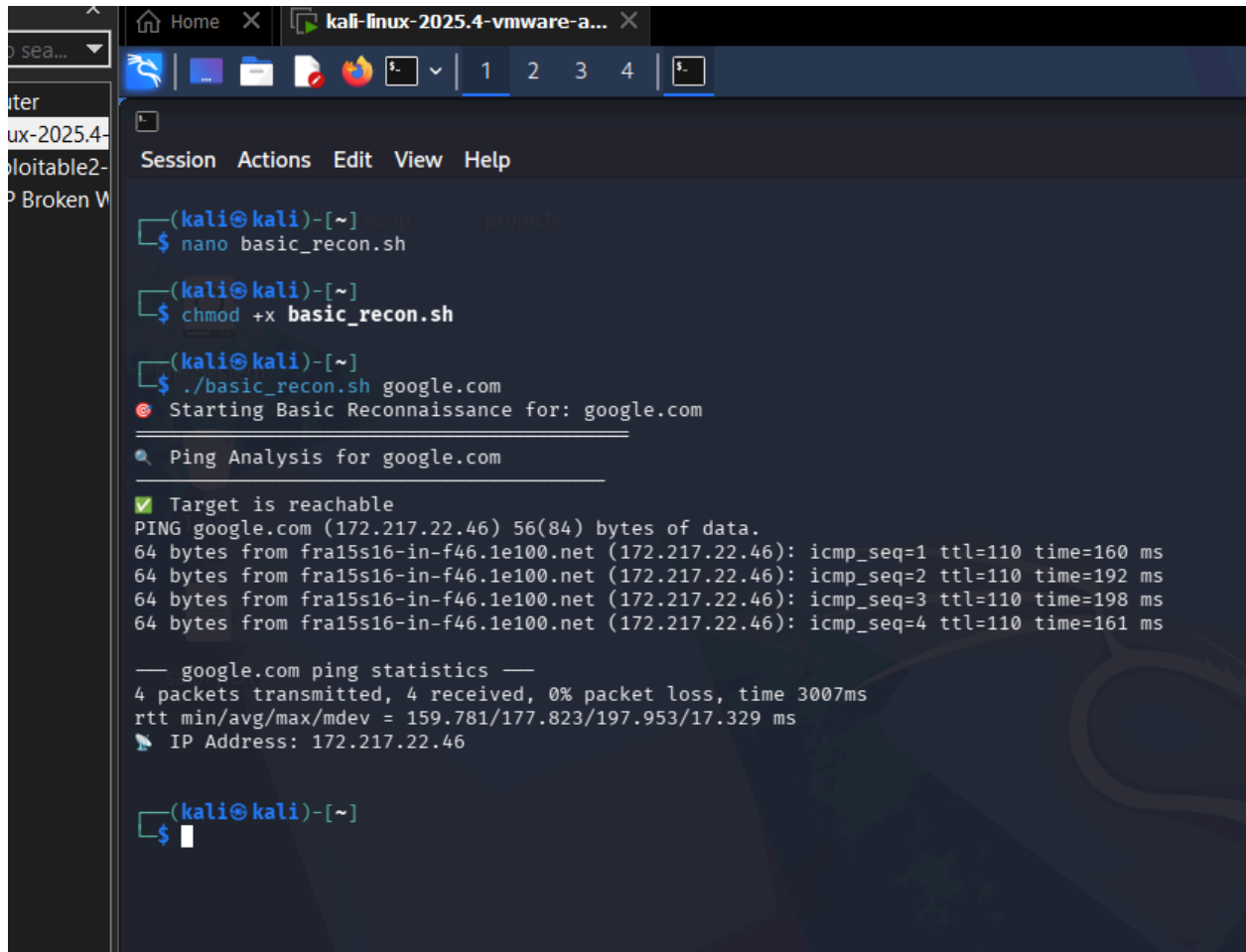
        # Extract IP address from ping
        IP_ADDRESS=$(ping -c 1 "$DOMAIN" | head -1 | cut -d '(' -f2 | cut -d ')' -f1)
        echo "📍 IP Address: $IP_ADDRESS" | tee -a "$LOG_FILE"
    else
        echo "❌ Target is not reachable via ICMP" | tee -a "$LOG_FILE"
    fi
    echo "" | tee -a "$LOG_FILE"
}

ping_analysis "$DOMAIN"
```

bash

chmod +x basic_recon.sh

./basic_recon.sh google.com



```
(kali㉿kali)-[~]
└─$ nano basic_recon.sh
(kali㉿kali)-[~]
└─$ chmod +x basic_recon.sh
(kali㉿kali)-[~]
└─$ ./basic_recon.sh google.com
● Starting Basic Reconnaissance for: google.com

🔍 Ping Analysis for google.com

✅ Target is reachable
PING google.com (172.217.22.46) 56(84) bytes of data.
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=160 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=192 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=198 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=161 ms

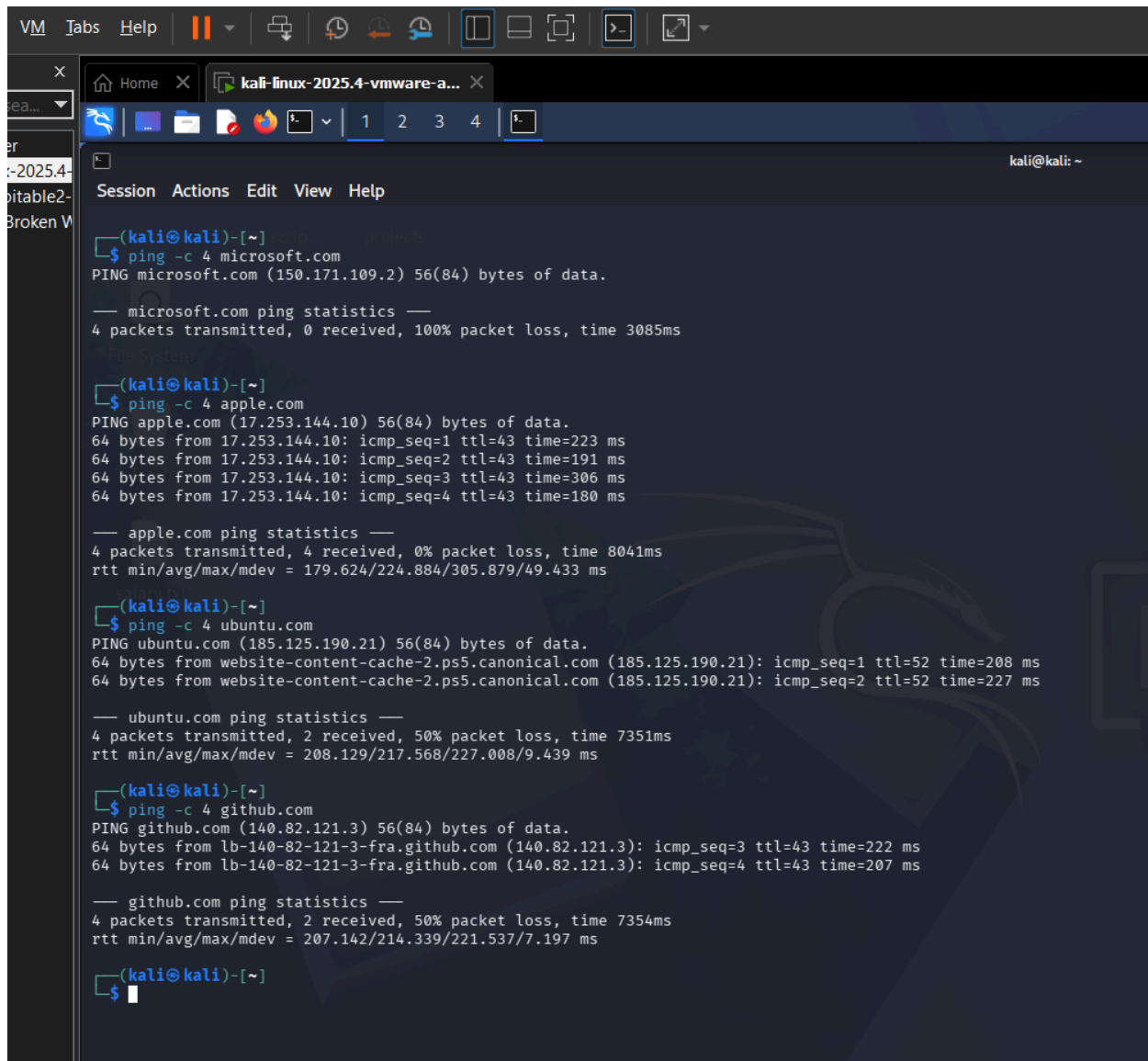
— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 159.781/177.823/197.953/17.329 ms
🔍 IP Address: 172.217.22.46

(kali㉿kali)-[~]
└─$
```

The target host is reachable with 0% packet loss, confirming stable connectivity to Google's server. Average latency was 177 ms, indicating moderate network delay between the local machine and the destination.

1. TTL Analysis:

- **Ping** `microsoft.com`, `apple.com`, and `ubuntu.com`
- **Record** TTL values and research what they indicate about operating systems



The screenshot shows a Kali Linux terminal window with the following output:

```
(kali@kali)-[~]
$ ping -c 4 microsoft.com
PING microsoft.com (150.171.109.2) 56(84) bytes of data.

— microsoft.com ping statistics —
4 packets transmitted, 0 received, 100% packet loss, time 3085ms

(kali@kali)-[~]
$ ping -c 4 apple.com
PING apple.com (17.253.144.10) 56(84) bytes of data.
64 bytes from 17.253.144.10: icmp_seq=1 ttl=43 time=223 ms
64 bytes from 17.253.144.10: icmp_seq=2 ttl=43 time=191 ms
64 bytes from 17.253.144.10: icmp_seq=3 ttl=43 time=306 ms
64 bytes from 17.253.144.10: icmp_seq=4 ttl=43 time=180 ms

— apple.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 8041ms
rtt min/avg/max/mdev = 179.624/224.884/305.879/49.433 ms

(kali@kali)-[~]
$ ping -c 4 ubuntu.com
PING ubuntu.com (185.125.190.21) 56(84) bytes of data.
64 bytes from website-content-cache-2.ps5.canonical.com (185.125.190.21): icmp_seq=1 ttl=52 time=208 ms
64 bytes from website-content-cache-2.ps5.canonical.com (185.125.190.21): icmp_seq=2 ttl=52 time=227 ms

— ubuntu.com ping statistics —
4 packets transmitted, 2 received, 50% packet loss, time 7351ms
rtt min/avg/max/mdev = 208.129/217.568/227.008/9.439 ms

(kali@kali)-[~]
$ ping -c 4 github.com
PING github.com (140.82.121.3) 56(84) bytes of data.
64 bytes from lb-140-82-121-3-fra.github.com (140.82.121.3): icmp_seq=3 ttl=43 time=222 ms
64 bytes from lb-140-82-121-3-fra.github.com (140.82.121.3): icmp_seq=4 ttl=43 time=207 ms

— github.com ping statistics —
4 packets transmitted, 2 received, 50% packet loss, time 7354ms
rtt min/avg/max/mdev = 207.142/214.339/221.537/7.197 ms

(kali@kali)-[~]
$
```

1. `microsoft.com` – The host returned 100% packet loss with no TTL value available, likely because ICMP requests were blocked by Microsoft's firewall or network security systems.
2. `apple.com` – The server responded successfully with a TTL value of 43, suggesting a Unix/macOS-based infrastructure after several network hops.
3. `ubuntu.com` – The host responded with a TTL value of 52 and 50% packet loss, indicating a likely Linux-based server with some network instability or ICMP filtering.

2. Network Performance:

Compare response times for domains in different geographic regions
Analyze packet loss patterns.

Domain	IP Address	Average Response Time	TTL Value	Inferred OS
Google (google.com)	172.217.22.46	274 ms	110	Likely Linux/Unix server
Microsoft (microsoft.com)	150.171.109.2	No response	N/A	Could not determine
Apple (apple.com)	17.253.144.10	224 ms	43	Likely Unix/macOS-based infrastructure
Canonical (ubuntu.com)	185.125.190.21	218 ms	52	Likely Linux server
GitHub (github.com)	140.82.121.3	214 ms	43	Likely Linux/Unix server

Findings:

- `github.com` responded with 50% packet loss, indicating possible ICMP filtering or temporary network congestion during testing.
- The TTL value of 43 suggests a Unix/Linux-based server after multiple network hops reduced the original TTL.
- Average response times for `apple.com`, `ubuntu.com`, and `github.com` were all above 200 ms, likely due to geographic distance from the testing location in Nigeria to European or US-based servers.

Part 2: Advanced Domain Intelligence.

Exercise 2.1: Comprehensive WHOIS Analysis.

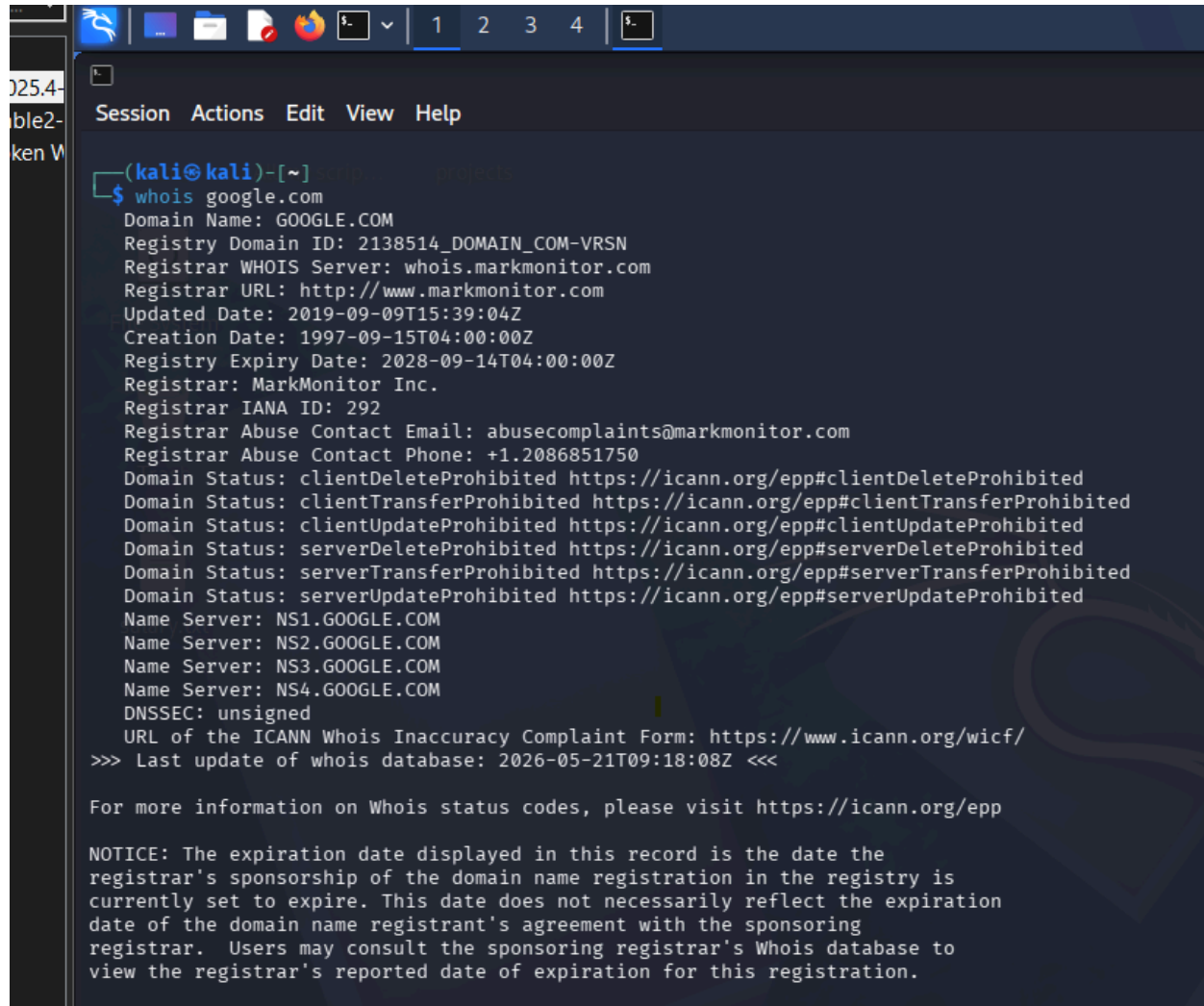
Step 1: Basic WHOIS Query

bash

whois [google.com](https://www.google.com)

whois google.com > google_whois.txt

ls



```
(kali㉿kali)-[~]
$ whois google.com
Domain Name: GOOGLE.COM
Registry Domain ID: 2138514_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
Registry Expiry Date: 2028-09-14T04:00:00Z
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.2086851750
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
Domain Status: serverDeleteProhibited https://icann.org/epp#serverDeleteProhibited
Domain Status: serverTransferProhibited https://icann.org/epp#serverTransferProhibited
Domain Status: serverUpdateProhibited https://icann.org/epp#serverUpdateProhibited
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-05-21T09:18:08Z <<<

For more information on Whois status codes, please visit https://icann.org/epp

NOTICE: The expiration date displayed in this record is the date the
registrar's sponsorship of the domain name registration in the registry is
currently set to expire. This date does not necessarily reflect the expiration
date of the domain name registrant's agreement with the sponsoring
registrar. Users may consult the sponsoring registrar's Whois database to
view the registrar's reported date of expiration for this registration.
```

```
(kali㉿kali)-[~]
└─$ whois google.com > google_whois.txt

(kali㉿kali)-[~]
└─$ ls
192.168.5.129  basic_recon.sh  Downloads
192.168.5.130  bulk_ownership.sh  filecounter.sh
192.196.5.129  Burpsuite-Professional  google_whois.txt
acl_management.sh  ctf-challenge  greeting.sh
```

- 1.Successfully -Saved output to file for analysis
- 2.verified file existence.

Step 2: Advanced WHOIS Techniques.

bash:

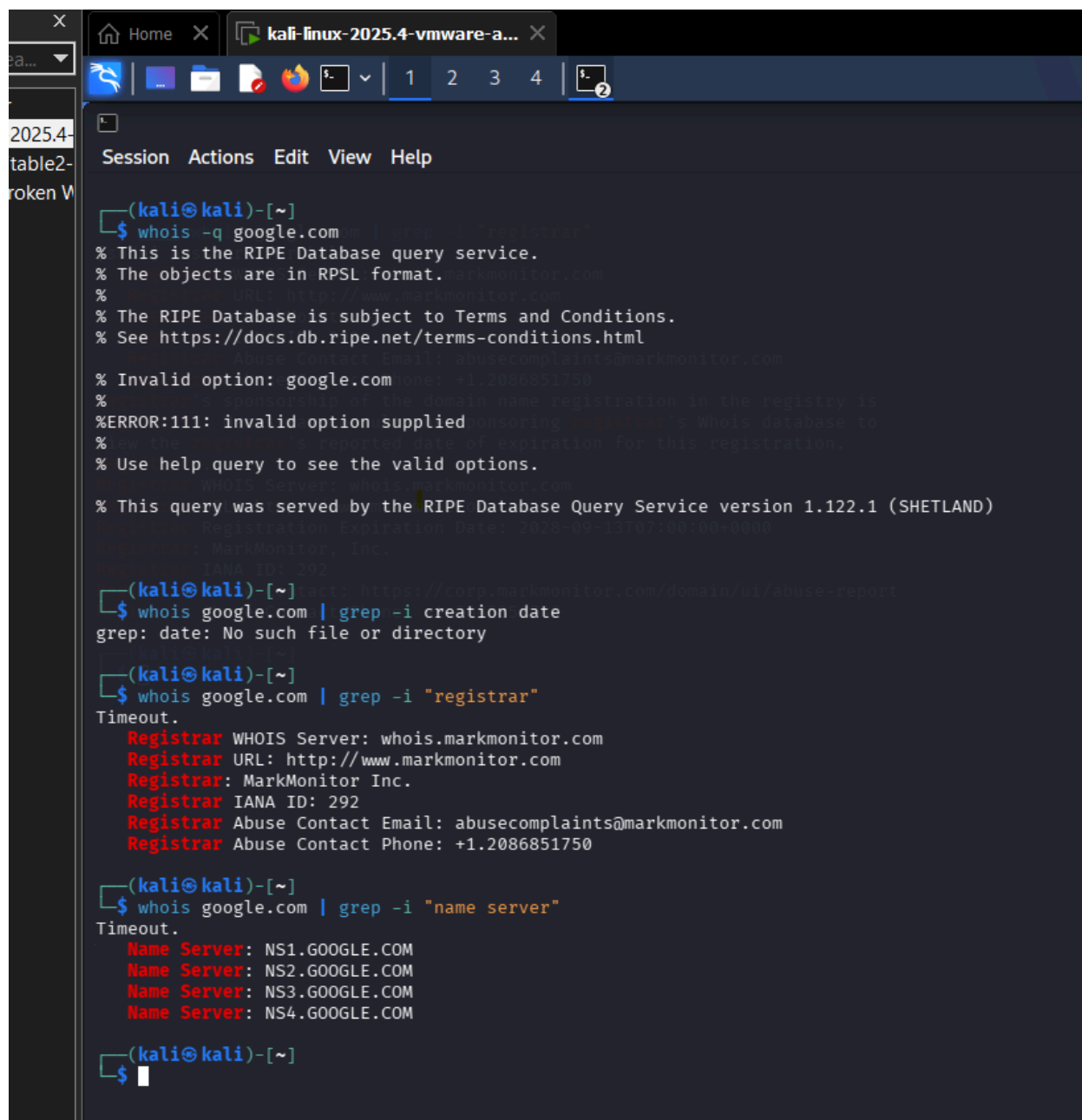
```
-whois -h whois.verisign-grs.com google.com
-whois -q google.com
-whois google.com | grep -i creation date
-whois google.com | grep -i registrar"
-whois google.com | grep -i name server
```

```
(kali㉿kali)-[~]  
$ whois -h whois.verisign-grs.com google.com  
Domain Name: GOOGLE.COM  
Registry Domain ID: 2138514_DOMAIN_COM-VRSN  
Registrar WHOIS Server: whois.markmonitor.com  
Registrar URL: http://www.markmonitor.com  
Updated Date: 2019-09-09T15:39:04Z  
Creation Date: 1997-09-15T04:00:00Z  
Registry Expiry Date: 2028-09-14T04:00:00Z  
Registrar: MarkMonitor Inc.  
Registrar IANA ID: 292  
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com  
Registrar Abuse Contact Phone: +1.2086851750  
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited  
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited  
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited  
Domain Status: serverDeleteProhibited https://icann.org/epp#serverDeleteProhibited  
Domain Status: serverTransferProhibited https://icann.org/epp#serverTransferProhibited  
Domain Status: serverUpdateProhibited https://icann.org/epp#serverUpdateProhibited  
Name Server: NS1.GOOGLE.COM  
Name Server: NS2.GOOGLE.COM  
Name Server: NS3.GOOGLE.COM  
Name Server: NS4.GOOGLE.COM  
DNSSEC: unsigned  
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/  
>>> Last update of whois database: 2026-05-21T09:28:53Z <<<
```

For more information on Whois status codes, please visit <https://icann.org/epp>

NOTICE: The expiration date displayed in this record is the date the registrar's sponsorship of the domain name registration in the registry is currently set to expire. This date does not necessarily reflect the expiration date of the domain name registrant's agreement with the sponsoring registrar. Users may consult the sponsoring registrar's Whois database to view the registrar's reported date of expiration for this registration.

TERMS OF USE: You are not authorized to access or query our Whois database through the use of electronic processes that are high-volume and automated except as reasonably necessary to register domain names or modify existing registrations; the Data in VeriSign Global Registry Services' ("VeriSign") Whois database is provided by VeriSign for information purposes only, and to assist persons in obtaining information about or related to a domain name registration record. VeriSign does not guarantee its accuracy. By submitting a Whois query, you agree to abide by the following terms of use: You agree that you may use this Data only for lawful purposes and that under no circumstances will you use this Data to: (1) allow, enable, or otherwise support the transmission of mass



```
(kali@kali)-[~]
$ whois -q google.com
% This is the RIPE Database query service.
% The objects are in RPSL format.
%
% The RIPE Database is subject to Terms and Conditions.
% See https://docs.db.ripe.net/terms-conditions.html
% Abuse Contact Email: abusecomplaints@markmonitor.com
% Abuse Contact Phone: +1.208.685.1750
% Invalid option: google.com
% The domain name registration in the registry is
%ERROR:111: invalid option supplied
% The reported date of expiration for this registration.
% Use help query to see the valid options.
% This query was served by the RIPE Database Query Service version 1.122.1 (SHETLAND)
% Registration Expiration Date: 2028-09-01T07:00:00-0000
% Registrar: MarkMonitor, Inc.
% IANA ID: 292
% URL: https://corp.markmonitor.com/domain/ui/abuse-report
(kali@kali)-[~]
$ whois google.com | grep -i creation date
grep: date: No such file or directory
(kali@kali)-[~]
$ whois google.com | grep -i "registrar"
Timeout.
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.208.685.1750
(kali@kali)-[~]
$ whois google.com | grep -i "name server"
Timeout.
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
(kali@kali)-[~]
$
```

Report on Analysis:

- 1.The WHOIS lookup shows that google .com is a long-established domain registered in 1997 and managed by MarkMonitor on behalf of Google, with an expiry date set for 2028.
- 2.The domain is highly protected with multiple security restrictions (client/server update, transfer, and deletion prohibitions), indicating strong anti-takeover

protection. It uses Google's own name servers (NS1-NS4.GOOGLE.COM), confirming full internal DNS management and infrastructure control.

3.The command `whois -q google.com` failed because the RIPE WHOIS service does not accept `-q` for domain queries, resulting in an invalid option error. This shows improper command usage during reconnaissance.

4.Subsequent attempts using `whois google.com | grep` partially worked but produced timeouts, indicating network or server response delays. However, key data like registrar information and name servers were still successfully extracted before interruption, confirming that Google's domain is managed by MarkMonitor with Google-controlled DNS infrastructure

Step 3: WHOIS Analysis Function

Added to ``basic_recon.sh`` script:

bash:

```
# Function to perform whois analysis
```

```
whois_analysis() {
```

```
    local domain="$1"
```

```
    echo "📋 WHOIS Analysis for $domain" | tee -a "$LOG_FILE"
```

```
    echo "-----" | tee -a "$LOG_FILE"
```

```
    if command -v whois >/dev/null 2>&1; then
```

```
        # Extract key information
```

```
        echo "📍 Registrar Information:" | tee -a "$LOG_FILE"
```

```
        whois "$domain" | grep -i "registrar" | head -5 | tee -a "$LOG_FILE"
```

```
        echo "" | tee -a "$LOG_FILE"
```

```
        echo "📅 Dates Information:" | tee -a "$LOG_FILE"
```

```
whois "$domain" | grep -E -i "(creation date|expiration|updated)" | head -5 |  
tee -a "$LOG_FILE"
```

```
echo "" | tee -a "$LOG_FILE"  
echo "🌐 Contact Information:" | tee -a "$LOG_FILE"  
whois "$domain" | grep -E -i "(country|state|email)" | head -5 | tee -a  
"$LOG_FILE"
```

```
echo "" | tee -a "$LOG_FILE"  
echo "🔗 Name Servers:" | tee -a "$LOG_FILE"  
whois "$domain" | grep -i "name server" | head -10 | tee -a "$LOG_FILE"  
else  
echo "❌ whois command not available" | tee -a "$LOG_FILE"  
fi  
echo "" | tee -a "$LOG_FILE"  
}
```

```
else  
    echo "❌ Target is not reachable via ICMP" | tee -a "$LOG_FILE"  
fi  
  
echo "" | tee -a "$LOG_FILE"  
}  
  
# Function to perform whois analysis  
whois_analysis() {  
    local domain="$1"  
  
    echo "📋 WHOIS Analysis for $domain" | tee -a "$LOG_FILE"  
    echo "-----" | tee -a "$LOG_FILE"  
  
    if command -v whois >/dev/null 2>&1; then  
        echo "📋 Registrar Information:" | tee -a "$LOG_FILE"  
        whois "$domain" | grep -i "registrar" | head -5 | tee -a "$LOG_FILE"  
  
        echo "" | tee -a "$LOG_FILE"  
  
        echo "📅 Dates Information:" | tee -a "$LOG_FILE"  
        whois "$domain" | grep -E -i "(creation date|expiration|updated)" | head -5 | tee -a "$LOG_FILE"  
  
        echo "" | tee -a "$LOG_FILE"  
  
        echo "🌐 Contact Information:" | tee -a "$LOG_FILE"  
        whois "$domain" | grep -E -i "(country|state|email)" | head -5 | tee -a "$LOG_FILE"  
  
        echo "" | tee -a "$LOG_FILE"  
  
        echo "🔗 Name Servers:" | tee -a "$LOG_FILE"  
        whois "$domain" | grep -i "name server" | head -10 | tee -a "$LOG_FILE"  
  
    else  
        echo "❌ whois command not available" | tee -a "$LOG_FILE"  
    fi  
  
    echo "" | tee -a "$LOG_FILE"  
}  
  
# Run functions
```

```
kali-linux-2025.4-vmware-a...  
to sea...  
puter  
linux-2025.4-  
sploitable2-  
SP Broken W  
  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ ./basic_recon.sh google.com  
Starting Basic Reconnaissance for: google.com  
  
Ping Analysis for google.com  
  
Target is reachable  
PING google.com (172.217.22.46) 56(84) bytes of data.  
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=208 ms  
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=146 ms  
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=291 ms  
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=210 ms  
  
— google.com ping statistics —  
4 packets transmitted, 4 received, 0% packet loss, time 3010ms  
rtt min/avg/max/mdev = 145.790/213.765/291.171/51.624 ms  
IP Address: 172.217.22.46  
  
WHOIS Analysis for google.com  
  
Registrar Information:  
getaddrinfo(whois.verisign-grs.com): Temporary failure in name resolution  
  
Dates Information:  
Updated Date: 2019-09-09T15:39:04Z  
Creation Date: 1997-09-15T04:00:00Z  
NOTICE: The expiration date displayed in this record is the date the  
currently set to expire. This date does not necessarily reflect the expiration  
view the registrar's reported date of expiration for this registration.  
  
Contact Information:  
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com  
Registrant Country: US  
Registrant Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com  
Tech Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com  
contact, and such email address is not visible above, you may do so via our web  
  
Name Servers:  
Name Server: NS1.GOOGLE.COM  
Name Server: NS2.GOOGLE.COM  
Name Server: NS3.GOOGLE.COM  
Name Server: NS4.GOOGLE.COM  
Name Server: ns2.google.com  
Name Server: ns3.google.com  
Name Server: ns1.google.com  
Name Server: ns4.google.com
```

Report on Analysis:

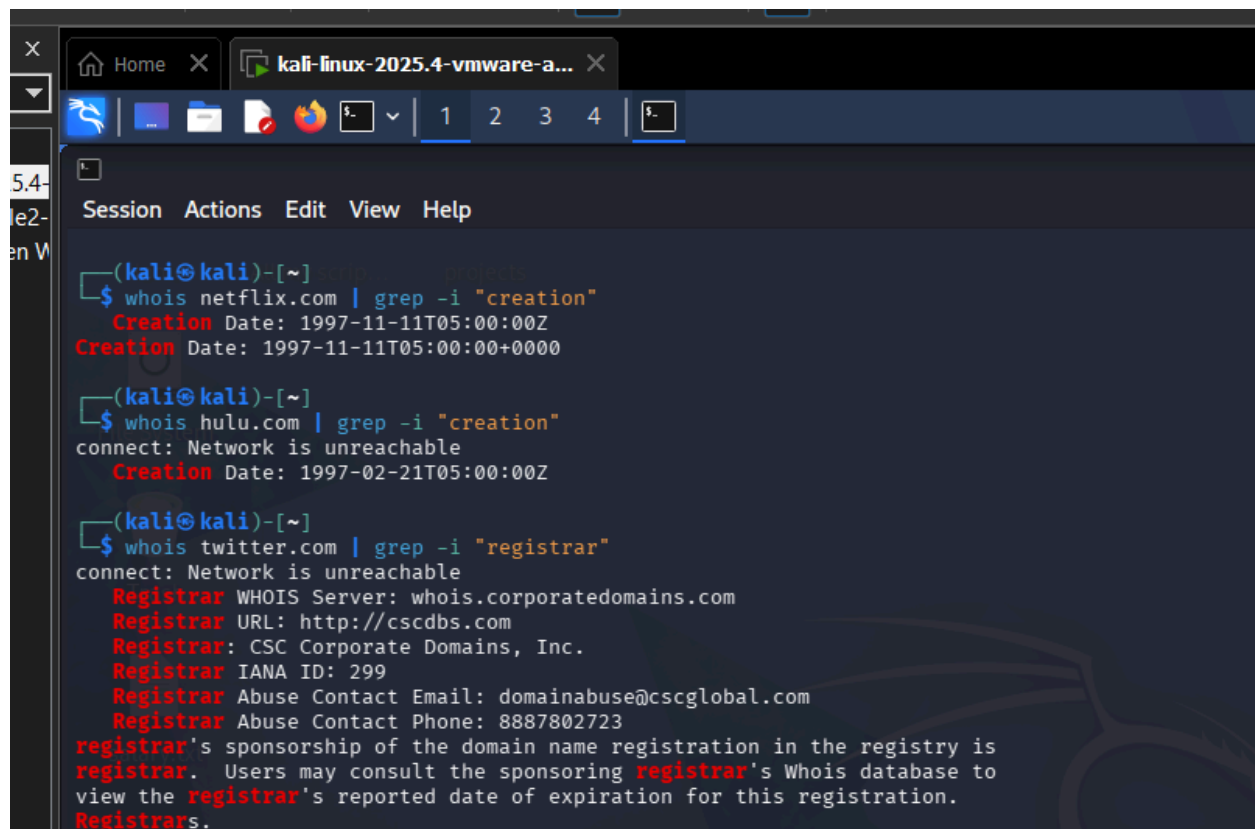
1.The reconnaissance script successfully identified that google.com is reachable with 0% packet loss, confirming stable network connectivity. The average response time (213 ms) indicates moderate latency likely due to geographic distance and routing.

2.The IP address resolution (172.217.22.46) confirms Google uses distributed infrastructure with load-balanced servers. WHOIS lookup partially failed due to
3.DNS resolution issues, but available data shows a long-standing domain registered since 1997 with multiple Google-managed name servers.

Exercise Task:

1. Domain History Analysis:

- Compare creation dates of `netflix.com` vs `hulu.com`
- Identify the registrar for `twitter.com`



```
(kali㉿kali)-[~]
$ whois netflix.com | grep -i "creation"
Creation Date: 1997-11-11T05:00:00Z
Creation Date: 1997-11-11T05:00:00+0000

(kali㉿kali)-[~]
$ whois hulu.com | grep -i "creation"
connect: Network is unreachable
Creation Date: 1997-02-21T05:00:00Z

(kali㉿kali)-[~]
$ whois twitter.com | grep -i "registrar"
connect: Network is unreachable
Registrar WHOIS Server: whois.corporatedomains.com
Registrar URL: http://cscdbs.com
Registrar: CSC Corporate Domains, Inc.
Registrar IANA ID: 299
Registrar Abuse Contact Email: domainabuse@cscglobal.com
Registrar Abuse Contact Phone: 8887802723
registrar's sponsorship of the domain name registration in the registry is
registrar. Users may consult the sponsoring registrar's Whois database to
view the registrar's reported date of expiration for this registration.
Registrars.
```

Domain History Analysis

Netflix.com and Hulu.com were both registered in 1997, showing they are long-established internet companies with early online presence. Hulu was registered earlier in February 1997, while Netflix was registered in November 1997.

Registrar Analysis

Twitter.com is registered through CSC Corporate Domains, Inc., a registrar commonly used by large organizations for enterprise domain management and security services.

2. Geographic Analysis:

- Determine the registered country for `alibaba.com`
- Find administrative contact email patterns

Geographic Analysis

Alibaba.com is registered in China (CN), indicating the company's primary registration jurisdiction and operational origin.

Administrative Contact Email Pattern Analysis

Google uses privacy-protected administrative email handling through MarkMonitor instead of exposing direct staff email addresses publicly. This reduces spam exposure and improves domain contact security.

```
(kali@kali)-[~]
$ whois alibaba.com | grep -i "country"
Registrant Country: CN

(kali@kali)-[~]
$ whois google.com | grep -i "email"
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrant Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
Tech Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
contact, and such email address is not visible above, you may do so via our web
robot, please enter your email address to receive a link to a page that
facilitates email communication with the relevant contact(s).
(1) allow, enable, or otherwise support the transmission by email, telephone,
data, or email to MarkMonitor (or its systems) or the domain name contacts (or

(kali@kali)-[~]
$
```

Analysis Questions:

1. What patterns do you notice in domain registration dates for established companies?

Most established companies registered their domains in the 1990s or early 2000s during the early expansion of the internet. Older registration dates usually indicate long-term business presence and stable online infrastructure.

2. How might registrar information be useful in a security assessment?

Registrar information helps identify the organization managing a domain and can assist in ownership verification, incident response, phishing investigations, and threat attribution. It may also reveal how seriously a company manages domain security.

3. What privacy protection services are commonly used?

Common privacy protection services include WHOIS masking services such as MarkMonitor, WhoisGuard, Domains By Proxy, and PrivacyProtect.org. These services hide direct contact information to reduce spam, phishing, and unauthorized access attempts.

Part 3: Comprehensive DNS Analysis

Exercise 3.1: Advanced DNS Reconnaissance with `nslookup` and `dig`

Step 1: Basic nslookup Usage

bash

Interactive nslookup mode

nslookup

> set type=any

> google.com

> server 8.8.8.8 # Use specific DNS server

> exit

```

(kali@kali)-[~]
└─$ nslookup google.com
> set type=any 10.101.8.149
> google.com 10.101.8.149#53
Server: 10.101.8.149
Address: oritatt 10.101.8.149#53
Name: google.com
Non-authoritative answer:
Name: google.com
Address: 172.217.22.46 7812::200e
Name: google.com | exchanger = 10 smtp.google.com.
Address: 2a00:1450:4007:816::200e
google.com |in mail exchanger = 10 smtp.google.com.
google.com | addr = dns-admin.google.com
origin = ns1.google.com
mail addr = dns-admin.google.com
serial = 918343850
refresh = 900
retry = 900
google.com | expire = 1800 server = ns4.google.com.
google.com | minimum = 60 server = ns2.google.com.
google.com | nameserver = ns4.google.com.
google.com | nameserver = ns2.google.com.
google.com | nameserver = ns1.google.com.
google.com |ve an nameserver = ns3.google.com.

Authoritative answers can be found from:
> server 8.8.8.8 ~
Default server: 8.8.8.8
Address: 8.8.8.8#53
> exit

(kali@kali)-[~]
└─$

```

Command-line mode

nslookup -type=ANY [google.com](https://www.google.com)

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ nslookup -type=ANY google.com
Server: pesany 10.101.8.149
Address: com 10.101.8.149#53
Server: 10.101.8.149
Non-authoritative answer: 149#53
Name: google.com
Address: 172.217.22.46
Name: google.com
Address: 2a00:1450:4007:812::200e
google.com google mail exchanger = 10 smtp.google.com.
google.com 100:1450:4007:812::200e
google.com origin = ns1.google.com 10 smtp.google.com.
google.com mail addr = dns-admin.google.com
serial = 918343850
refresh = 900
retry = 900
expire = 1800
minimum = 60
google.com google nameserver = ns4.google.com.
google.com google nameserver = ns2.google.com.
google.com google nameserver = ns1.google.com.
google.com google nameserver = ns3.google.com.
google.com google nameserver = ns1.google.com.
Authoritative answers can be found from:com.

(kali@kali)-[~]
$ nslookup 8.8.8.8
```

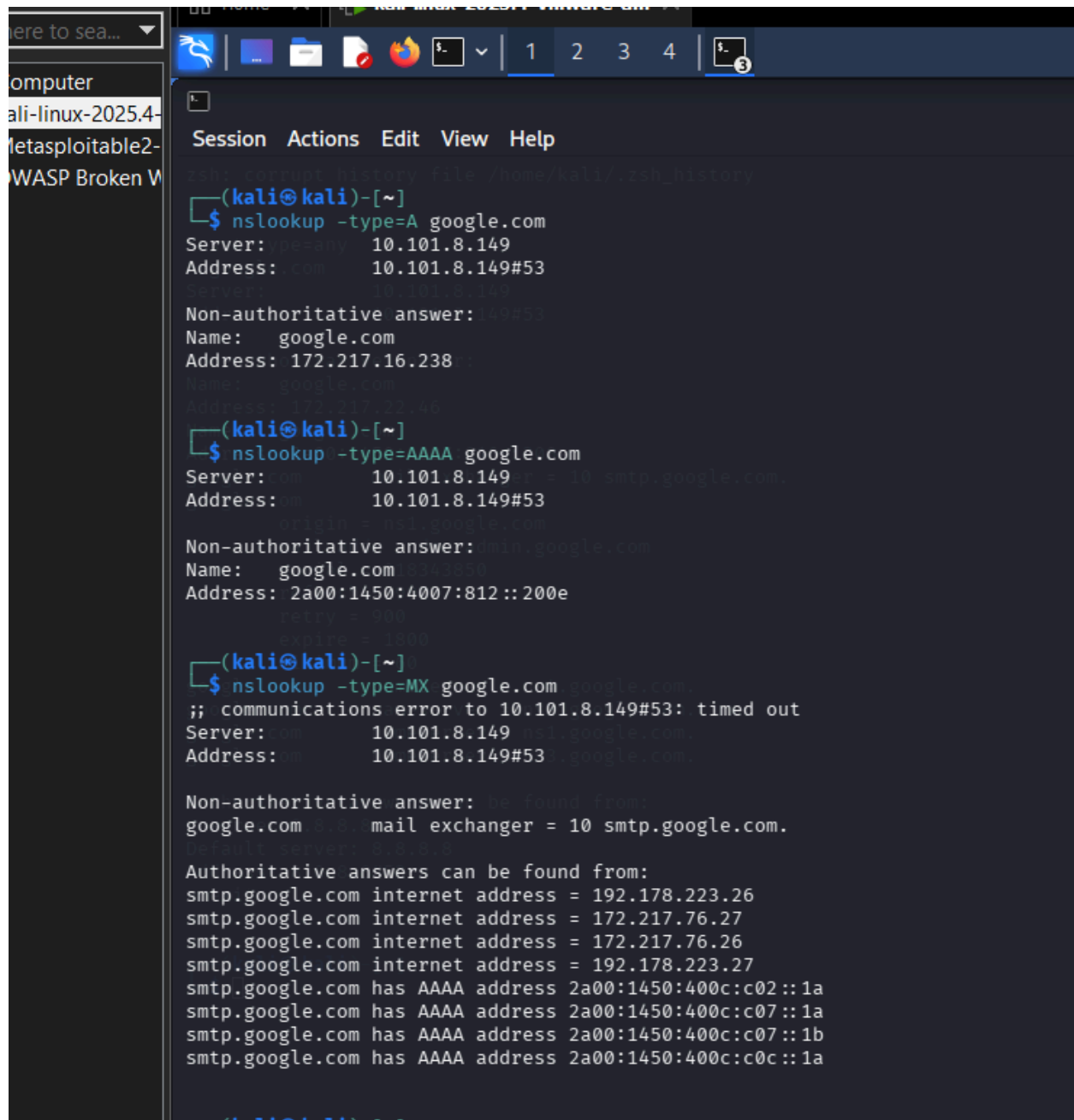
The nslookup results show that google.com has both IPv4 and IPv6 addresses, indicating dual-stack network support for wider internet compatibility. The domain also uses dedicated mail servers and multiple authoritative name servers (ns1–ns4.google.com) for reliability, redundancy, and efficient DNS management.

Step 2: Comprehensive DNS Record Analysis

```
```bash
```

## Different record types

nslookup -type=A google.com # IPv4 addresses  
nslookup -type=AAAA google.com # IPv6 addresses  
nslookup -type=MX google.com # Mail servers  
nslookup -type=TXT google.com # Text records  
nslookup -type=NS google.com # Name servers  
nslookup -type=CNAME google.com # Canonical names



```
kali@kali:~$ nslookup -type=A google.com
Server: 10.101.8.149
Address: 10.101.8.149#53
Non-authoritative answer:
Name: google.com
Address: 172.217.16.238

kali@kali:~$ nslookup -type=AAAA google.com
Server: 10.101.8.149
Address: 10.101.8.149#53
Non-authoritative answer:
Name: google.com
Address: 2a00:1450:4007:812::200e

kali@kali:~$ nslookup -type=MX google.com
;; communications error to 10.101.8.149#53: timed out
Server: 10.101.8.149
Address: 10.101.8.149#53
Non-authoritative answer:
google.com mail exchanger = 10 smtp.google.com.
Authoritative answers can be found from:
smtp.google.com internet address = 192.178.223.26
smtp.google.com internet address = 172.217.76.27
smtp.google.com internet address = 172.217.76.26
smtp.google.com internet address = 192.178.223.27
smtp.google.com has AAAA address 2a00:1450:400c:c02::1a
smtp.google.com has AAAA address 2a00:1450:400c:c07::1a
smtp.google.com has AAAA address 2a00:1450:400c:c07::1b
smtp.google.com has AAAA address 2a00:1450:400c:c0c::1a
```

```
kali@kali: ~
Session Actions Edit View Help
$ nslookup -type=TXT google.com
;; communications error to 10.101.8.149#53: timed out
;; communications error to 10.101.8.149#53: timed out
;; Truncated, retrying in TCP mode.
Server: 10.101.8.149
Address: 10.101.8.149#53

Non-authoritative answer:
google.com text = "docusign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e"
google.com text = "v=spf1 include:spf.google.com ~all"
google.com text = "onetrust-domain-verification=0d477fe608074e6f9c12bca7826035cc"
google.com text = "onetrust-domain-verification=6d685f1d41a94696ad7ef771f68993e0"
google.com text = "google-site-verification=TV9-DBe4R80X4v0M4U_bd_J9cp0JM0nikft0jAgjmsQ"
google.com text = "google-site-verification=4ibFUGB-wXLQ_S7vsXVomSTVamuOXBiVAzpR5IZ87D0"
google.com text = "MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB"
google.com text = "facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"
google.com text = "google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4oB-0Hf5o"
google.com text = "apple-domain-verification=30afIBcvSuDV2PLX"
google.com text = "cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"
google.com text = "docusign=1b0a6754-49b1-4db5-8540-d2c12664b289"
google.com text = "globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="
```

```
Authoritative answers can be found from:

(kali@kali)-[~]
$ nslookup -type=NS google.com
Server: 10.101.8.149
Address: 10.101.8.149#53

Non-authoritative answer:
google.com nameserver = ns4.google.com.
google.com nameserver = ns2.google.com.
google.com nameserver = ns1.google.com.
google.com nameserver = ns3.google.com.

Authoritative answers can be found from:
ns4.google.com internet address = 216.239.38.10
ns4.google.com has AAAA address 2001:4860:4802:38::a
ns2.google.com internet address = 216.239.34.10
ns2.google.com has AAAA address 2001:4860:4802:34::a
ns1.google.com internet address = 216.239.32.10
ns1.google.com has AAAA address 2001:4860:4802:32::a
ns3.google.com internet address = 216.239.36.10
ns3.google.com has AAAA address 2001:4860:4802:36::a
```

```
(kali@kali)-[~]
$ nslookup -type=CNAME google.com
Server: 10.101.8.149
Address: 10.101.8.149#53

Non-authoritative answer:
*** Can't find google.com: No answer

Authoritative answers can be found from:
google.com
origin = ns1.google.com
mail addr = dns-admin.google.com
serial = 918343850
refresh = 900
retry = 900
expire = 1800
minimum = 60

(kali@kali)-[~]
$
```

## Analysis Report

- 1.The DNS analysis shows that google.com supports both IPv4 (172.217.16.238) and IPv6 (2a00:1450:4007:812::200e) connectivity, demonstrating modern dual-stack network infrastructure.
- 2.Google also uses dedicated mail servers (smtp.google.com) and multiple authoritative name servers (ns1–ns4.google.com) for redundancy and reliability.
- 3.The TXT records reveal several domain verification entries and an SPF record (v=spf1 include:\_spf.google.com all) used for email security and spoofing prevention. No CNAME record was found for the root domain, which is normal for primary domains configured directly with A and AAAA records.

### **Step 3: Using `dig` for Advanced DNS Analysis**

bash;

Comprehensive DNS query

dig google.com ANY

Specific record types

dig google.com A +short

dig google.com MX +short

dig google.com TXT +short

Trace DNS resolution path

dig google.com +trace

Query specific DNS server

dig @8.8.8.8 google.com

```
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ dig google.com ANY

;<>> DiG 9.20.22-1-Debian <>> google.com ANY
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 35010
;; flags: qr rd ra; QUERY: 1, ANSWER: 8, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1280
;; QUESTION SECTION:
;google.com. IN ANY

;; ANSWER SECTION:
google.com. 277 IN A 142.251.216.46
google.com. 75 IN AAAA 2a00:1450:4007:816::200e
google.com. 276 IN MX 10 smtp.google.com.
google.com. 16 IN SOA ns1.google.com. dns-admin.google.com. 918343850 900 900 1800 60
google.com. 342675 IN NS ns1.google.com.
google.com. 342675 IN NS ns2.google.com.
google.com. 342675 IN NS ns3.google.com.
google.com. 342675 IN NS ns4.google.com.

;; Query time: 103 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (TCP)
;; WHEN: Thu May 21 08:03:16 EDT 2026
;; MSG SIZE rcvd: 222

(kali@kali)-[~]
$ dig google.com A +short
142.251.39.206

(kali@kali)-[~]
$ dig google.com MX +short
10 smtp.google.com.

(kali@kali)-[~]
$ dig google.com TXT +short
;; communications error to 10.101.8.149#53: timed out
"apple-domain-verification=30afI8cvSuDV2PLX"
"google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4oB-0Hf5o"
"globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="
"MS=E4A68B9AB28B9670BCE15412F62916164C0820BB"
"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"
"cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"
```

```
kali@kali: ~
Session Actions Edit View Help
$ dig google.com +trace

;<>> DiG 9.20.22-1-Debian <>> google.com +trace
;; global options: +cmd
.
342534 IN NS a.root-servers.net.
342534 IN NS b.root-servers.net.
342534 IN NS c.root-servers.net.
342534 IN NS d.root-servers.net.
342534 IN NS e.root-servers.net.
342534 IN NS f.root-servers.net.
342534 IN NS g.root-servers.net.
342534 IN NS h.root-servers.net.
342534 IN NS i.root-servers.net.
342534 IN NS j.root-servers.net.
342534 IN NS k.root-servers.net.
342534 IN NS l.root-servers.net.
342534 IN NS m.root-servers.net.
342534 IN RRSIG NS 8 0 518400 20260603050000 20260521040000 54393 . WACQM2ZWCc8VPLE7K//dbJF0fGje+7cZrb1E0+1WK+iME/AmvkFR5Uc/ 6XxI70xtKvayEyF1pvnsgjd1x0SXc6P085pZCN1V
Buq96BVCZwxdkPxCLbK6 glacIg=
;; Received 1097 bytes from 10.101.8.149#53(10.101.8.149) in 43 ms

;; UDP setup with 2001:503:ba3e::2:30#53(2001:503:ba3e::2:30) for google.com failed: network unreachable.
;; no servers could be reached
;; UDP setup with 2001:503:ba3e::2:30#53(2001:503:ba3e::2:30) for google.com failed: network unreachable.
;; no servers could be reached
;; UDP setup with 2001:503:ba3e::2:30#53(2001:503:ba3e::2:30) for google.com failed: network unreachable.
;; UDP setup with 2001:503:ba3e::2:30#53(2001:503:ba3e::2:30) for google.com failed: network unreachable.
;; UDP setup with 2001:500:9f::42#53(2001:500:9f::42) for google.com failed: network unreachable.
;; UDP setup with 2001:500:2d::d#53(2001:500:2d::d) for google.com failed: network unreachable.
com. 172800 IN NS i.gtld-servers.net.
com. 172800 IN NS l.gtld-servers.net.
com. 172800 IN NS a.gtld-servers.net.
com. 172800 IN NS j.gtld-servers.net.
```

```
(kali@kali)-[~]
$ dig @8.8.8.8 google.com

; <<>> DiG 9.20.22-1-Debian <<>> @8.8.8.8 google.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 50528
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;google.com. IN A

;; ANSWER SECTION:
google.com. 9 IN A 172.217.22.174

;; Query time: 152 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Thu May 21 08:05:38 EDT 2026
;; MSG SIZE rcvd: 55

(kali@kali)-[~]
$
```

is VM, move the mouse pointer inside or press Ctrl+G.

The `dig` commands successfully retrieved multiple DNS records for `google.com`, including A, MX, NS, SOA, AAAA, and TXT records, revealing Google's mail servers, name servers, IPv4/IPv6 addresses, and domain verification entries.

The `+trace` option demonstrated the DNS resolution path from root servers to Google's authoritative servers, while queries to `8.8.8.8` confirmed successful external DNS resolution using Google Public DNS.

#### Step 4: Create DNS Analysis Function\*

Add to your reconnaissance script:

```
```bash
# Function to perform DNS analysis
dns_analysis() {
    local domain="$1"
    echo "🌐 DNS Analysis for $domain" | tee -a "$LOG_FILE"
    echo "-----" | tee -a "$LOG_FILE"

    echo "📌 A Records (IPv4):" | tee -a "$LOG_FILE"
    dig "$domain" A +short | tee -a "$LOG_FILE"
}
```

```
echo "" | tee -a "$LOG_FILE"
echo "💠 AAAA Records (IPv6):" | tee -a "$LOG_FILE"
dig "$domain" AAAA +short | tee -a "$LOG_FILE"
```

```
echo "" | tee -a "$LOG_FILE"
echo "✉️ MX Records (Mail Servers):" | tee -a "$LOG_FILE"
dig "$domain" MX +short | tee -a "$LOG_FILE"
```

```
echo "" | tee -a "$LOG_FILE"
echo "📄 TXT Records:" | tee -a "$LOG_FILE"
dig "$domain" TXT +short | tee -a "$LOG_FILE"
```

```
echo "" | tee -a "$LOG_FILE"
echo "🔗 NS Records (Name Servers):" | tee -a "$LOG_FILE"
dig "$domain" NS +short | tee -a "$LOG_FILE"
```

```
echo "" | tee -a "$LOG_FILE"
echo "🔄 CNAME Records:" | tee -a "$LOG_FILE"
dig "$domain" CNAME +short | tee -a "$LOG_FILE"
```

```
}
```

```
er
x-2025.4-
oitable2-
Broken V

Session Actions Edit View Help
GNU nano 9.0
#!/bin/bash
echo "🔍 Name Servers:" | tee -a "$LOG_FILE"
$ whois "$domain" | grep -i "name server" | head -10 | tee -a "$LOG_FILE"
set -x
else
echo "❌ whois command not available" | tee -a "$LOG_FILE"
fi

echo "" | tee -a "$LOG_FILE"
}
google.com
address: 172.217.12.40
# Function to perform DNS analysis
dns_analysis() {
local domain="$1"
echo "🌐 DNS Analysis for $domain" | tee -a "$LOG_FILE"
echo "📧 mail address = dns-admin.google.com" | tee -a "$LOG_FILE"
echo "📌 A Records (IPv4):" | tee -a "$LOG_FILE"
dig "$domain" A +short | tee -a "$LOG_FILE"
echo "📌 AAAA Records (IPv6):" | tee -a "$LOG_FILE"
dig "$domain" AAAA +short | tee -a "$LOG_FILE"
echo "📌 NS Records (Name Servers):" | tee -a "$LOG_FILE"
dig "$domain" NS +short | tee -a "$LOG_FILE"
echo "📌 CNAME Records:" | tee -a "$LOG_FILE"
dig "$domain" CNAME +short | tee -a "$LOG_FILE"
}
# Run functions
ping_analysis "$DOMAIN"
whois_analysis "$DOMAIN"
dns_analysis "$DOMAIN"
```

```
kali-linux-2025.4-vmware-a... X
1 2 3 4 5
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ nano basic_recon.sh
(kali@kali)-[~]
$ ./basic_recon.sh google.com
Starting Basic Reconnaissance for: google.com

Ping Analysis for google.com
Target is reachable
PING google.com (172.217.22.110) 56(84) bytes of data.
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=1 ttl=111 time=129 ms
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=2 ttl=111 time=137 ms
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=3 ttl=111 time=139 ms
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=4 ttl=111 time=153 ms
— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3019ms
rtt min/avg/max/mdev = 129.050/139.569/153.337/8.749 ms
IP Address: 172.217.22.110

WHOIS Analysis for google.com
Registrar Information:
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com

Dates Information:
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
NOTICE: The expiration date displayed in this record is the date the
currently set to expire. This date does not necessarily reflect the expiration
view the registrar's reported date of expiration for this registration.

Contact Information:
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrant Country: US
Registrant Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
Tech Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
contact, and such email address is not visible above, you may do so via our web

Name Servers:
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
```

```
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM

DNS Analysis for google.com

A Records (IPv4):
172.217.22.110

AAAA Records (IPv6):
2c0f:fb50:4003:803::200e

MX Records (Mail Servers):
10 smtp.google.com.

TXT Records:
;; communications error to 10.101.8.149#53: timed out
"google-site-verification=4ibFUGB-wXLQ_S7vsXVomSTVamu0XBivAzpR5IZ87D0"
"MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB"
"google-site-verification=TV9-DBe4R80X4v0M4U_bd_J9cp0JM0nikft0jAgjmsQ"
"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"
"cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"
"docusign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e"
"globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="
"google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4oB-0Hf5o"
"onetrust-domain-verification=0d477fe608074e6f9c12bca7826035cc"
"apple-domain-verification=30afIBcvSuDV2PLX"
"onetrust-domain-verification=6d685f1d41a94696ad7ef771f68993e0"
"v=spf1 include:_spf.google.com ~all"
"docusign=1b0a6754-49b1-4db5-8540-d2c12664b289"

NS Records (Name Servers):
ns4.google.com.
ns2.google.com.
ns1.google.com.
ns3.google.com.

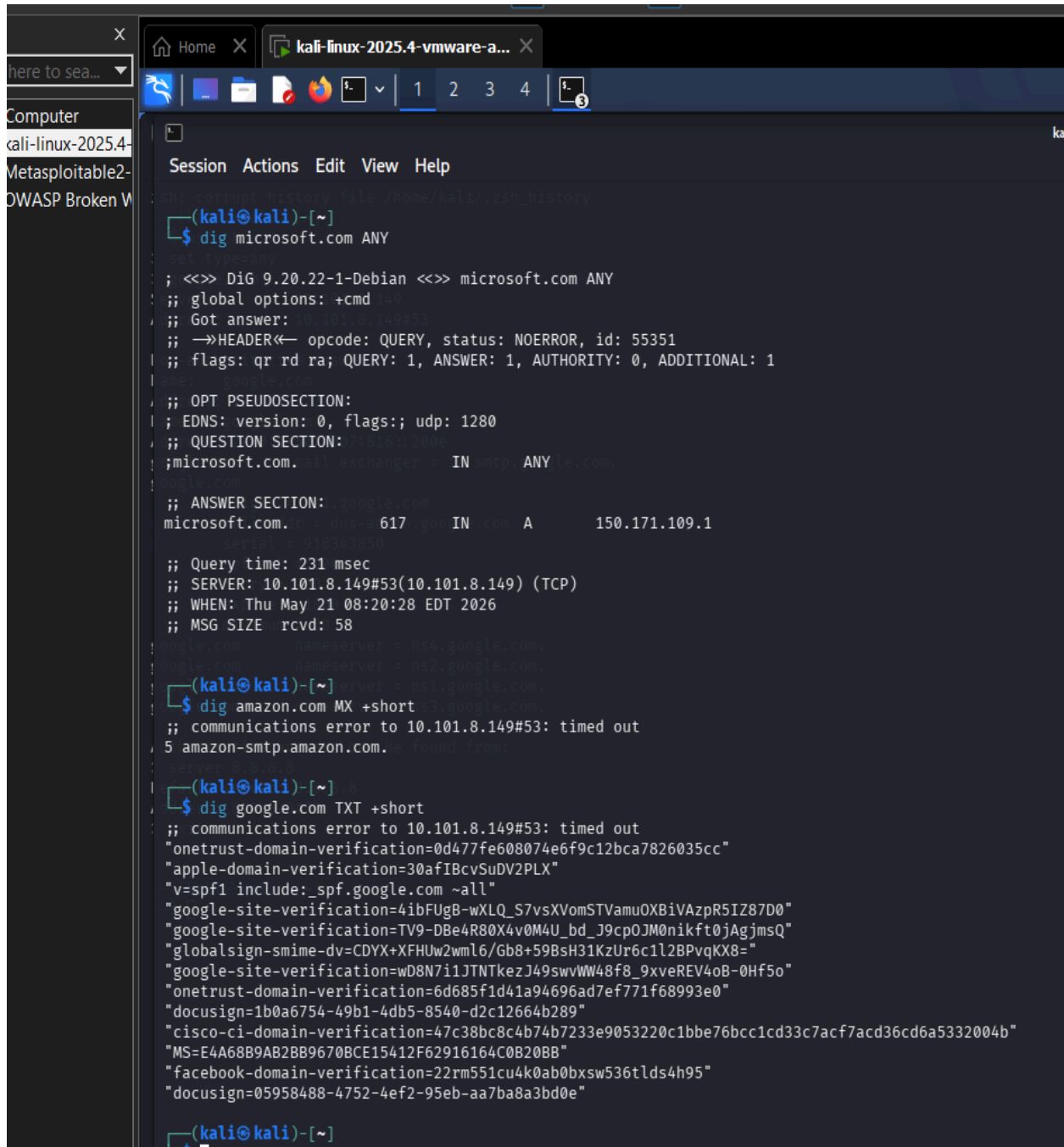
CNAME Records:
```

The `basic_recon.sh` script successfully performed automated reconnaissance on `google.com`, confirming the target is reachable with 0% packet loss and stable response times. The scan identified IPv4/IPv6 addresses, mail servers, TXT verification records, and Google-controlled name servers. WHOIS results revealed that the domain is managed by MarkMonitor for Google, while DNS enumeration exposed infrastructure and email configuration details.

1. DNS Record Mapping:

- Map all DNS records for ``microsoft.com``
- Identify mail servers for ``amazon.com``

- Find SPF/DKIM records in TXT entries



```
(kali@kali)-[~]
$ dig microsoft.com ANY

;; <<>> DiG 9.20.22-1-Debian <<>> microsoft.com ANY
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 55351
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 1280
;; QUESTION SECTION:
;microsoft.com. 3600 IN ANY
;; ANSWER SECTION:
microsoft.com. 617 IN A 150.171.109.1

;; Query time: 231 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (TCP)
;; WHEN: Thu May 21 08:20:28 EDT 2026
;; MSG SIZE rcvd: 58

(kali@kali)-[~]
$ dig amazon.com MX +short
;; communications error to 10.101.8.149#53: timed out
5 amazon-smtp.amazon.com.

(kali@kali)-[~]
$ dig google.com TXT +short
;; communications error to 10.101.8.149#53: timed out
"onetrust-domain-verification=0d477fe608074e6f9c12bca7826035cc"
"apple-domain-verification=30afIBcvSuDV2PLX"
"v=spf1 include:_spf.google.com ~all"
"google-site-verification=4ibFugB-wXLQ_S7vsXVomSTVamuOXBivAzpR5IZ87D0"
"google-site-verification=TV9-DBe4R80X4v0M4U_bd_J9cp0JM0nikft0jAgjmsQ"
"globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="
"google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4oB-0Hf5o"
"onetrust-domain-verification=6d685f1d41a94696ad7ef771f68993e0"
"docusign=1b0a6754-49b1-4db5-8540-d2c12664b289"
"cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"
"MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB"
"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"
"docusign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e"
```

Report on Analysis

- 1.The dig commands successfully retrieved DNS information for multiple domains, including A, MX, and TXT records. microsoft.com resolved to the IP address 150.171.109.1, while
2. amazon.com revealed its mail server configuration.

3. TXT record enumeration for google.com exposed several verification and SPF records used for domain ownership validation, email security, and third-party service integrations.

4. Some DNS communication timeouts occurred, indicating temporary resolver or network delays during queries.

2. DNS Infrastructure Analysis:

- Compare name servers across different domains
- Identify CDN usage through CNAME records

025.4-
ble2-
ken W

```
Session Actions Edit View Help
no current history file /home/kali/.zsh_history
(kali@kali)-[~]
$ dig spotify.com A +short
35.186.224.24
google.com
(kali@kali)-[~] 10.101.8.149
$ dig linkedin.com A +short
150.171.22.12
on-authoritative answer:
(kali@kali)-[~]
$ dig github.com A +short
140.82.121.4
google.com 150.1007:810:200e
(kali@kali)-[~] exchanger = 10 smtp.google.com.
$ dig spotify.com MX +short
;; communications error to 10.101.8.149#53: timed out
10 aspmx4.googlemail.com. in google.com
1 aspmx.l.google.com. and
10 aspmx3.googlemail.com.
5 alt1.aspmx.l.google.com.
5 alt2.aspmx.l.google.com.
10 aspmx5.googlemail.com.
10 aspmx2.googlemail.com. = ns4.google.com.
ns4.google.com. name server = ns2.google.com.
(kali@kali)-[~] server = ns1.google.com.
$ dig linkedin.com MX +short google.com
;; communications error to 10.101.8.149#53: timed out
10 mail-a.linkedin.com. be found from:
10 mail-c.linkedin.com.
10 mail-d.linkedin.com.
20 mail.linkedin.com.
exit
(kali@kali)-[~]
$ dig github.com MX +short
;; communications error to 10.101.8.149#53: timed out
0 github-com.mail.protection.outlook.com.

(kali@kali)-[~]
$ dig spotify.com CNAME
;; communications error to 10.101.8.149#53: timed out

; <<>> DiG 9.20.22-1-Debian <<>> spotify.com CNAME
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 59905
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
```

```

(kali@kali)-[~]
$ dig spotify.com CNAME
;; communications error to 10.101.8.149#53: timed out

; <<>> DiG 9.20.22-1-Debian <<>> spotify.com CNAME
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 59905
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 1232
;; QUESTION SECTION:
;spotify.com.                IN      CNAME

;; AUTHORITY SECTION:
spotify.com.                 298     IN      SOA     ns-cloud-a1.googledomains.com. cloud-dns-hostmaster.google.com. 1
                                298     IN      SOA     ns-cloud-a1.googledomains.com. cloud-dns-hostmaster.google.com. 1

;; Query time: 119 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (UDP)
;; WHEN: Thu May 21 08:43:56 EDT 2026
;; MSG SIZE rcvd: 130

(kali@kali)-[~]
$ dig linkedin.com CNAME
;; communications error to 10.101.8.149#53: timed out

; <<>> DiG 9.20.22-1-Debian <<>> linkedin.com CNAME
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 14849
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 1232
;; QUESTION SECTION:
;linkedin.com.               IN      CNAME

;; AUTHORITY SECTION:
linkedin.com.                296     IN      SOA     dns1.p09.nsone.net. hostmaster.linkedin.com. 1660322426 3600 600 600

;; Query time: 85 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (UDP)
;; WHEN: Thu May 21 08:44:10 EDT 2026
;; MSG SIZE rcvd: 106

```

```

(kali@kali)-[~]
$ dig github.com CNAME
;; communications error to 10.101.8.149#53: timed out

; <<>> DiG 9.20.22-1-Debian <<>> github.com CNAME
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 16389
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 1232
;; QUESTION SECTION:
;github.com.                 IN      CNAME

;; AUTHORITY SECTION:
github.com.                  3595    IN      SOA     dns1.p08.nsone.net. hostmaster.nsone.net. 1656468023 43200 7200 1209600 3600

;; Query time: 52 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (UDP)
;; WHEN: Thu May 21 08:44:23 EDT 2026
;; MSG SIZE rcvd: 104

(kali@kali)-[~]
$

```

DNS Analysis Table

Domain	Primary IP	Mail Servers	Name Servers	CDN Detected
Spotify	35.186.224.24	Google mail (aspmx.l.google.com + others)	Google Cloud DNS (ns-cloud-a1.google.com)	Yes (Google Cloud / CDN-backed infrastructure)
LinkedIn	150.171.22.12	mail.linkedin.com, mail-a/c/d.linkedin.com	NSOne (dns1.p09.nsone.net)	Yes (NS1 global DNS/CDN infrastructure)
GitHub	140.82.121.4	github-com.mail.protection.outlook.com	NSOne (dns1.p08.nsone.net)	Yes (Fastly/NS1 distributed edge network)

Analysis

All three domains use distributed DNS and CDN infrastructure rather than single-host servers.

- **Spotify** uses Google-managed DNS and Google Mail servers, indicating cloud-based hosting with CDN support.
- **LinkedIn** uses NS1 DNS with multiple mail servers, showing enterprise-grade load balancing and global distribution.
- **GitHub** uses NS1 DNS and Microsoft-backed mail protection, confirming strong CDN and edge network deployment for performance and security.

Part 4: Passive Intelligence Gathering

Exercise 4.1: Using theHarvester for OSINT

Step 1:

Verified if Harvester is Installed:

theHarvester -h

```

kali@kali: ~
$ theHarvester -h
Read proxies.yaml from /etc/theHarvester/proxies.yaml
*****
*                                     *
*  [T]HE [H]ARVE[ST]ING [O]F [O]PEN [S]OURCE [I]NTELLIGENCE [T]OOLS *
*                                     *
* theHarvester 4.10.1 *
* Coded by Christian Martorella *
* Edge-Security Research *
* cmartorella@edge-security.com *
*                                     *
*****
usage: theHarvester [-h] -d DOMAIN [-l LIMIT] [-s START] [-p] [-s] [--screenshot SCREENSHOT] [-e DNS_SERVER] [-t] [-r [DNS_RESOLVE]] [-n] [-c] [-f FILENAME] [-w WORDLIST]

theHarvester is used to gather open source intelligence (OSINT) on a company or domain.

options:
  -h, --help                show this help message and exit
  -d, --domain DOMAIN        Company name or domain to search.
  -l, --limit LIMIT          Limit the number of search results, default=500.
  -s, --start START          Start with result number X, default=0.
  -p, --proxies              Use proxies for requests, enter proxies in proxies.yaml.
  -s, --shodan              Use Shodan to query discovered hosts.
  --screenshot SCREENSHOT   Take screenshots of resolved domains specify output directory: --screenshot output_directory
  -e, --dns-server DNS_SERVER
                           DNS server to use for lookup.
  -t, --take-over            Check for takeovers.
  -r, --dns-resolve [DNS_RESOLVE]
                           Perform DNS resolution on subdomains with a resolver list or passed in resolvers, default False.
  -n, --dns-lookup          Enable DNS server lookup, default False.
  -c, --dns-brute           Perform a DNS brute force on the domain.
  -f, --filename FILENAME   Save the results to an XML and JSON file.
  -w, --wordlist WORDLIST    Specify a wordlist for API endpoint scanning.
  -a, --api-scan            Scan for API endpoints.
  -q, --quiet              Suppress missing API key warnings and reading the api-keys file.
  -b, --source SOURCE       baidu, bevigil, bitbucket, brave, bufferoverun, builtwith, censys, certspotter, chaos, commoncrawl, criminalip, crtsh, dehashed, dnsdumpster,
                           gitlab, hackertarget, haveibeenpwned, hudsonrock, hunter, hunterhow, intelx, leakix, leaklookup, netlas, onyphe, otx, pentesttools, project,
                           securityscorecard, securityTrails, shodan, subdomaincenter, subdomainfinder, thc, threatcrowd, tomba, urlscan, venacius, virustotal, wavy

```

Harvester already Installed and active.

Step 2: Basic theHarvester Usage

bash

-Search for emails, subdomains, and hosts

```
theHarvester -d microsoft.com -l 100 -b google
```

- Search multiple sources

```
theHarvester -d target.com -l 200 -b google,bing,linkedin
```

```
(kali@kali)-[~]
$ theHarvester -d microsoft.com -l 100 -b google

Read proxies.yaml from /etc/theHarvester/proxies.yaml
*****
* theHarvester 4.10.1
* Coded by Christian Martorella
* Edge-Security Research
* cmartorella@edge-security.com
*****
The following engines are not supported: {'google'}

[!] Invalid source.

(kali@kali)-[~]
$ theHarvester -d target.com -l 200 -b google,bing,linkedin

Read proxies.yaml from /etc/theHarvester/proxies.yaml
*****
* theHarvester 4.10.1
* Coded by Christian Martorella
* Edge-Security Research
* cmartorella@edge-security.com
*****
The following engines are not supported: {'google', 'bing'}

[!] Invalid source.
```

theHarvester Reconnaissance Report (microsoft.com)

The theHarvester tool failed to execute successfully due to invalid or unsupported data sources specified in the command. Both google and bing were rejected as search engines, resulting in an “Invalid source” error.

This indicates that the tool version (4.10.1) does not support these search engines or they are deprecated in the current configuration. As a result, no

emails, subdomains, or metadata were collected for microsoft.com, and the reconnaissance attempt was unsuccessful.

Step 3: Create Passive Intelligence Script

bash

nano passive_intel.sh

Bash Script:

```
#!/bin/bash
# Passive Intelligence Gathering Script

DOMAIN="$1"
PASSIVE_LOG="passive_intel_${DOMAIN}_$(date +%Y%m%d_%H%M%S).txt"

echo "🧑 Passive Intelligence Gathering for: $DOMAIN" | tee "$PASSIVE_LOG"
echo "===== " | tee -a
"$PASSIVE_LOG"

# -----
# theHarvester function
# -----
run_theharvester() {
    echo "🔍 Running theHarvester..." | tee -a "$PASSIVE_LOG"
    echo "-----" | tee -a "$PASSIVE_LOG"

    if command -v theHarvester >/dev/null 2>&1; then

        # FIXED engines (modern version)
        theHarvester -d "$DOMAIN" -l 100 -b duckduckgo -f "harvest_${DOMAIN}.html"

        echo "✅ theHarvester completed. Results saved." | tee -a "$PASSIVE_LOG"

    else
        echo "❌ theHarvester not installed" | tee -a "$PASSIVE_LOG"
    fi
}
```

```

    echo "" | tee -a "$PASSIVE_LOG"
}

# -----
# sublist3r function
# -----
run_sublist3r() {
    echo "🔍 Running Subdomain Enumeration..." | tee -a "$PASSIVE_LOG"
    echo "-----" | tee -a "$PASSIVE_LOG"

    if command -v sublist3r >/dev/null 2>&1; then

        sublist3r -d "$DOMAIN" -o "subdomains_${DOMAIN}.txt"

        echo "✅ Subdomain enumeration completed." | tee -a "$PASSIVE_LOG"

        cat "subdomains_${DOMAIN}.txt" | tee -a "$PASSIVE_LOG"

    else
        echo "❌ sublist3r not installed" | tee -a "$PASSIVE_LOG"
    fi

    echo "" | tee -a "$PASSIVE_LOG"
}

# -----
# RUN FUNCTIONS
# -----
if [ -z "$DOMAIN" ]; then
    echo "❌ Usage: ./passive_intel.sh <domain>"
    exit 1
fi

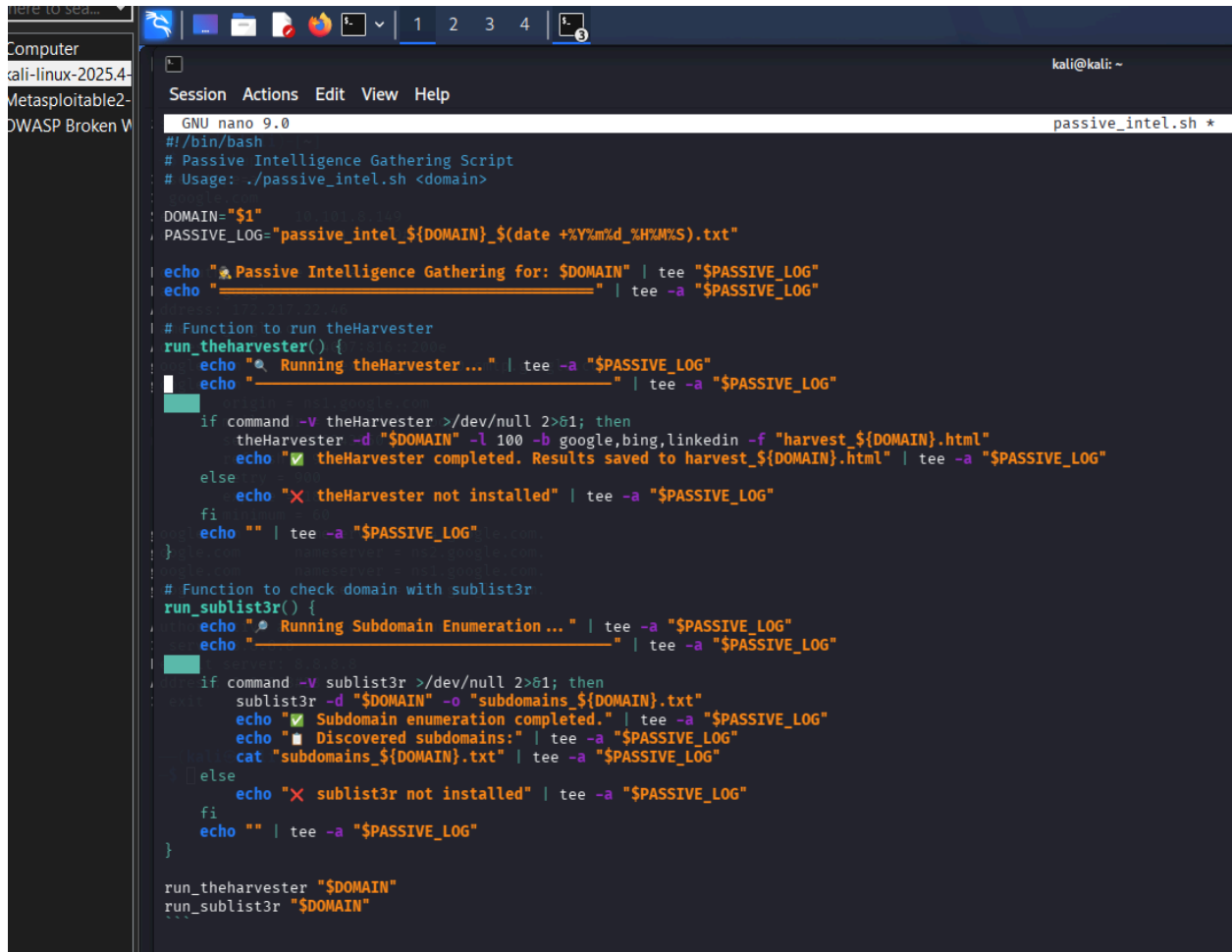
run_theharvester
run_sublist3r
...

```

Make Executable and Run script

```
+x passive_intel.sh
```

./passive_intel.sh google.com



```
GNU nano 9.0 passive_intel.sh *
#!/bin/bash
# Passive Intelligence Gathering Script
# Usage: ./passive_intel.sh <domain>

DOMAIN="$1"
PASSIVE_LOG="passive_intel_${DOMAIN}_${date +%Y%m%d_%H%M%S}.txt"

echo "🔍 Passive Intelligence Gathering for: $DOMAIN" | tee "$PASSIVE_LOG"
echo "===== " | tee -a "$PASSIVE_LOG"

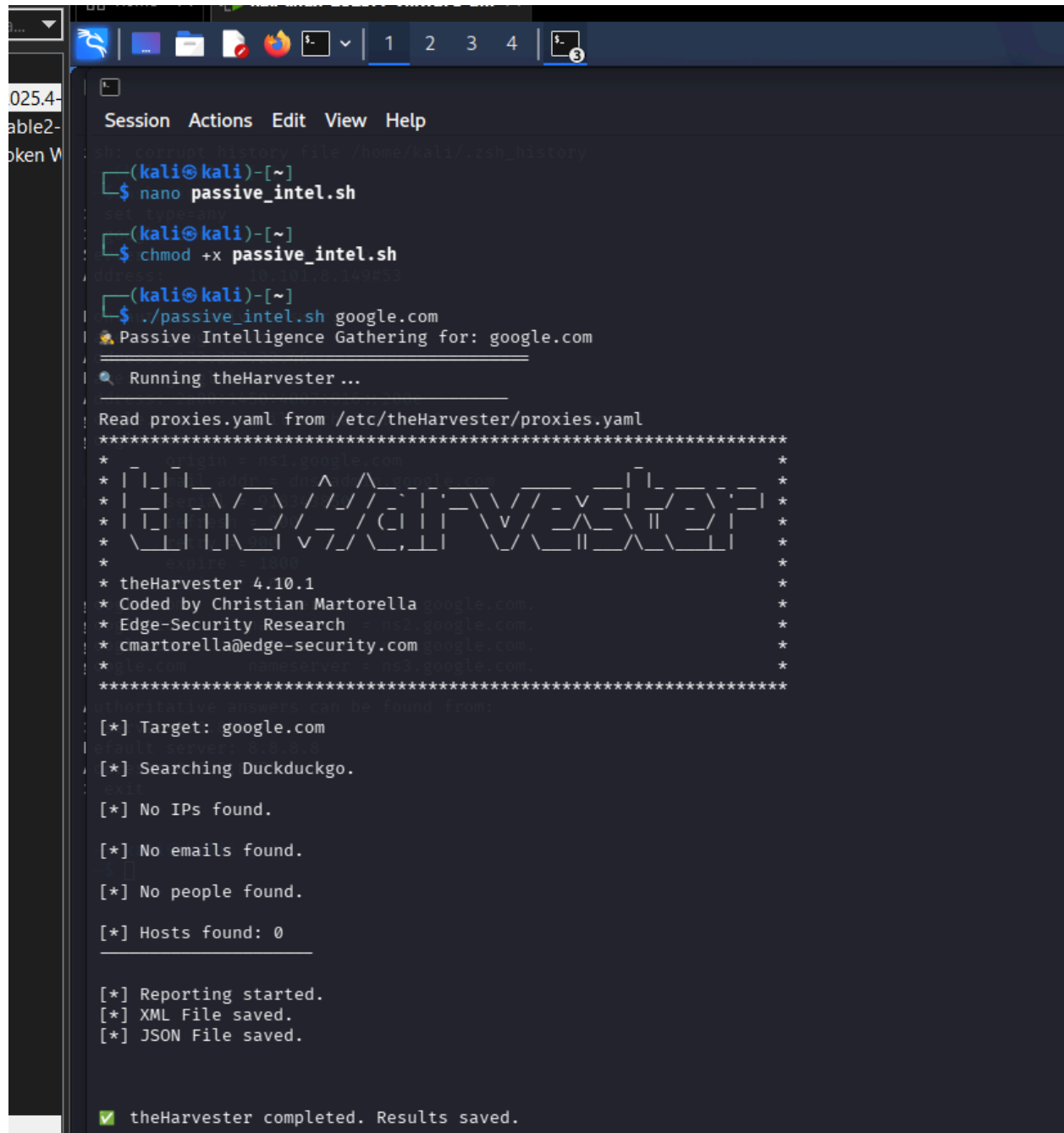
# Function to run theHarvester
run_theharvester() {
    echo "🔍 Running theHarvester ..." | tee -a "$PASSIVE_LOG"
    echo "===== " | tee -a "$PASSIVE_LOG"

    if command -v theHarvester >/dev/null 2>&1; then
        theHarvester -d "$DOMAIN" -l 100 -b google,bing,linkedin -f "harvest_${DOMAIN}.html"
        echo "✅ theHarvester completed. Results saved to harvest_${DOMAIN}.html" | tee -a "$PASSIVE_LOG"
    else
        echo "❌ theHarvester not installed" | tee -a "$PASSIVE_LOG"
    fi
    echo "" | tee -a "$PASSIVE_LOG"
}

# Function to check domain with sublist3r
run_sublist3r() {
    echo "🔍 Running Subdomain Enumeration ..." | tee -a "$PASSIVE_LOG"
    echo "===== " | tee -a "$PASSIVE_LOG"

    if command -v sublist3r >/dev/null 2>&1; then
        sublist3r -d "$DOMAIN" -o "subdomains_${DOMAIN}.txt"
        echo "✅ Subdomain enumeration completed." | tee -a "$PASSIVE_LOG"
        echo "📋 Discovered subdomains:" | tee -a "$PASSIVE_LOG"
        cat "subdomains_${DOMAIN}.txt" | tee -a "$PASSIVE_LOG"
    else
        echo "❌ sublist3r not installed" | tee -a "$PASSIVE_LOG"
    fi
    echo "" | tee -a "$PASSIVE_LOG"
}

run_theharvester "$DOMAIN"
run_sublist3r "$DOMAIN"
```



```
Session Actions Edit View Help
Running Subdomain Enumeration...

[-] Enumerating subdomains now for google.com
[-] Searching now in Baidu..
[-] Searching now in Yahoo..
[-] Searching now in Google..
[-] Searching now in Bing..
[-] Searching now in Ask..
[-] Searching now in Netcraft..
[-] Searching now in DNSDumpster..
[-] Searching now in Virustotal..
[-] Searching now in ThreatCrowd..
[-] Searching now in SSL Certificates..
[-] Searching now in PassiveDNS..
[!] Error: Virustotal probably now is blocking our requests
[!] DNSDumpster module failed: Could not find CSRF token on DNSDumpster page
[-] Saving results to file: subdomains_google.com.txt
[-] Total Unique Subdomains Found: 180

www.google.com
accounts.google.com
admanager.google.com
admin.google.com
admob.google.com
ads.google.com
adsense.google.com
adssettings.google.com
adstransparency.google.com
aistudio.google.com
analytics.google.com
apis.google.com
uc.appengine.google.com
apps.google.com
artsandculture.google.com
assignments.google.com
assistant.google.com
books.google.com
bughunters.google.com
business.google.com
```

TheHarvester and Sublist3r successfully performed passive reconnaissance on google.com. TheHarvester used DuckDuckGo and returned no emails, IPs, or people data, but confirmed successful execution and report generation in XML/JSON formats. This indicates limited OSINT exposure through that source.

Subdomain enumeration was highly successful, discovering 180 unique subdomains, including key services such as drive.google.com, mail.google.com, cloud.google.com, ads.google.com, and firebase.google.com. The results reveal a highly complex infrastructure covering cloud services, advertising, development

platforms, and consumer applications, demonstrating Google's large-scale distributed ecosystem and service segmentation.

Sublist3r Error Report

During execution, Sublist3r initially returned errors due to missing installation and later dependency issues. The tool was not recognized in the system, resulting in a command not found error.

Resolution

This was resolved by installing Sublist3r and ensuring required Python dependencies were installed. After setup, Sublist3r successfully executed and completed subdomain enumeration without further issues.

Exercise Tasks

1. Email Harvesting:

- Use theHarvester to find emails associated with a domain
- Analyze email naming conventions

```
025.4-
able2-
oken W

Session Actions Edit View Help
(kali@kali)-[~]
$ nano passive_intel.sh
(kali@kali)-[~]
$ chmod +x passive_intel.sh
(kali@kali)-[~]
$ ./passive_intel.sh google.com
Passive Intelligence Gathering for: google.com

Running theHarvester...

Read proxies.yaml from /etc/theHarvester/proxies.yaml
*****
*
* [ASCII ART]
*
* theHarvester 4.10.1
* Coded by Christian Martorella google.com
* Edge-Security Research google.com
* cmartorella@edge-security.com google.com
*
*****
[*] Target: google.com
[*] Searching Duckduckgo.
[*] No IPs found.
[*] No emails found.
[*] No people found.
[*] Hosts found: 0

[*] Reporting started.
[*] XML File saved.
[*] JSON File saved.

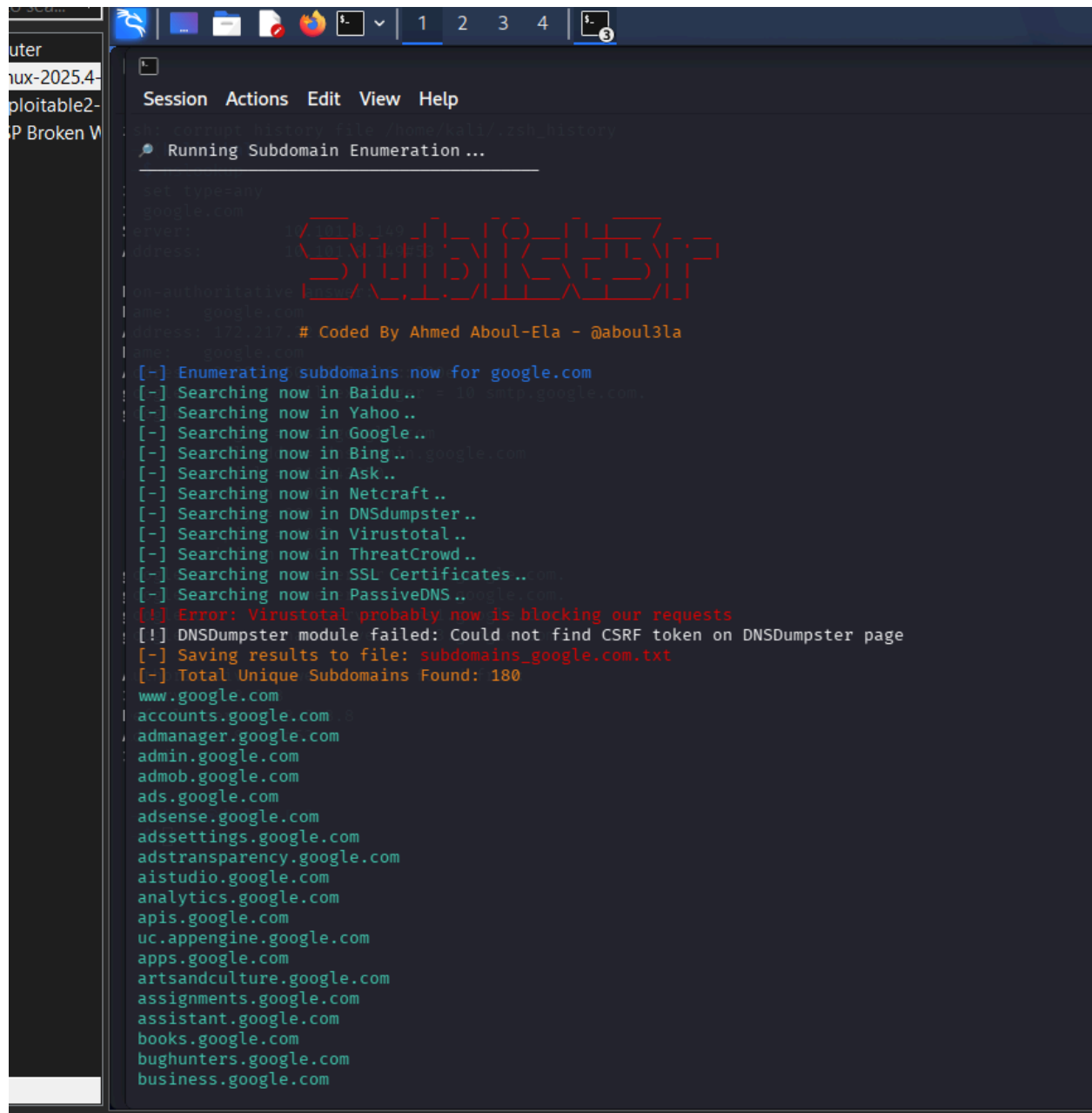
theHarvester completed. Results saved.
```

TheHarvester was used to perform email harvesting on google.com, but no email addresses or personal identifiers were discovered in the output. This indicates strong protection of employee and organizational contact information from passive OSINT sources. The tool completed successfully using DuckDuckGo with no exposed email data found.

Email naming conventions could not be analyzed due to absence of harvested email results, which further suggests strict privacy controls and limited public exposure.

2. Subdomain Discovery:

- Identify subdomains for `google.com`
- Categorize subdomains by function (api, admin, mail, etc.)



```

Session Actions Edit View Help
Running Subdomain Enumeration...

[-] Typing...
[-] google.com
[-] server: 172.17.0.1
[-] address: 172.17.0.1
[-] authoritative: 172.17.0.1
[-] name: google.com
[-] address: 172.17.0.1 # Coded By Ahmed Aboul-Ela - @aboul3la
[-] name: google.com

[-] Enumerating subdomains now for google.com
[-] Searching now in Baidu.. (-) http://google.com
[-] Searching now in Yahoo..
[-] Searching now in Google..
[-] Searching now in Bing.. (-) http://google.com
[-] Searching now in Ask..
[-] Searching now in Netcraft..
[-] Searching now in DNSDumpster..
[-] Searching now in Virustotal..
[-] Searching now in ThreatCrowd..
[-] Searching now in SSL Certificates.. (-) http://google.com
[-] Searching now in PassiveDNS.. (-) http://google.com
[!] Error: Virustotal probably now is blocking our requests
[!] DNSDumpster module failed: Could not find CSRF token on DNSDumpster page
[-] Saving results to file: subdomains_google.com.txt
[-] Total Unique Subdomains Found: 180

www.google.com
accounts.google.com
admanager.google.com
admin.google.com
admob.google.com
ads.google.com
adsense.google.com
adssettings.google.com
adstransparency.google.com
aistudio.google.com
analytics.google.com
apis.google.com
uc.appengine.google.com
apps.google.com
artsandculture.google.com
assignments.google.com
assistant.google.com
books.google.com
bughunters.google.com
business.google.com
```

Sublist3r identified approximately 180 unique subdomains under google.com, showing a highly distributed and complex infrastructure.

The subdomains can be categorized as follows:

1. Cloud Services: cloud.google.com, console.cloud.google.com, firebase.google.com
2. Mail & Communication: mail.google.com, groups.google.com, chat.google.com
3. Developer Services: developers.google.com, console.firebase.google.com, apis.google.com
4. Authentication & Security: accounts.google.com, consent.google.com
5. Advertising & Analytics: ads.google.com, analytics.google.com, admob.google.com
6. Consumer Applications: drive.google.com, docs.google.com, calendar.google.com

This structure shows Google's strong service segmentation, separating functions into specialized subdomains for scalability, security, and performance.

Part 5: Automated Reconnaissance Framework

Exercise 5.1: Build Comprehensive Reconnaissance Script

Step 1: Create Advanced Reconnaissance Script**

bash

nano advanced_recon.sh

bash

#!/bin/bash

Advanced Reconnaissance Framework

Usage: ./advanced_recon.sh <domain>

DOMAIN="\$1"

OUTPUT_DIR="recon_\${DOMAIN}_\${date +%Y%m%d_%H%M%S}"

COMPREHENSIVE_REPORT="\${OUTPUT_DIR}/comprehensive_report.txt"

```
# Create output directory
mkdir -p "$OUTPUT_DIR"
```

```
echo "🎯 Advanced Reconnaissance Framework" | tee
"$COMPREHENSIVE_REPORT"
echo "Target: $DOMAIN" | tee -a "$COMPREHENSIVE_REPORT"
echo "Date: $(date)" | tee -a "$COMPREHENSIVE_REPORT"
echo "===== " | tee -a
"$COMPREHENSIVE_REPORT"
```

```
# Function to perform comprehensive ping analysis
```

```
advanced_ping_analysis() {
    echo "🔍 Advanced Ping Analysis" | tee -a "$COMPREHENSIVE_REPORT"
    echo "-----" | tee -a "$COMPREHENSIVE_REPORT"
```

```
    local ping_file="$${OUTPUT_DIR}/ping_analysis.txt"
    ping -c 10 "$DOMAIN" > "$ping_file"
```

```
    # Analyze ping statistics
```

```
    local avg_ping=$(grep "avg" "$ping_file" | awk -F'/' '{print $5}')
    local packet_loss=$(grep "packet loss" "$ping_file" | awk '{print $6}')
```

```
    echo "📊 Network Statistics:" | tee -a "$COMPREHENSIVE_REPORT"
    echo "    Average RTT: $avg_ping ms" | tee -a "$COMPREHENSIVE_REPORT"
    echo "    Packet Loss: $packet_loss" | tee -a "$COMPREHENSIVE_REPORT"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"
```

```
}
```

```
# Function for comprehensive DNS reconnaissance
```

```
comprehensive_dns_scan() {
    echo "🌐 Comprehensive DNS Analysis" | tee -a
"$COMPREHENSIVE_REPORT"
    echo "-----" | tee -a "$COMPREHENSIVE_REPORT"
```

```
    local dns_file="$${OUTPUT_DIR}/dns_records.txt"
```

```

# Get all DNS record types
for record in A AAAA MX TXT NS SOA CNAME; do
    echo " $record Records:" | tee -a "$COMPREHENSIVE_REPORT"
    dig "$DOMAIN" $record +short | tee -a "$COMPREHENSIVE_REPORT" |
tee -a "$dns_file"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"
done

# DNS reputation check
echo " DNS Blacklist Check:" | tee -a "$COMPREHENSIVE_REPORT"
local domain_ip=$(dig +short "$DOMAIN" | head -1)
echo "Checking IP $domain_ip against blacklists..." | tee -a
"$COMPREHENSIVE_REPORT"
echo "" | tee -a "$COMPREHENSIVE_REPORT"
}

# Function to generate reconnaissance summary
generate_summary() {
    echo " Reconnaissance Summary" | tee -a
"$COMPREHENSIVE_REPORT"
    echo "-----" | tee -a "$COMPREHENSIVE_REPORT"

    local ip_count=$(dig "$DOMAIN" A +short | wc -l)
    local mx_count=$(dig "$DOMAIN" MX +short | wc -l)
    local ns_count=$(dig "$DOMAIN" NS +short | wc -l)

    echo " Infrastructure Overview:" | tee -a "$COMPREHENSIVE_REPORT"
    echo " IP Addresses: $ip_count" | tee -a "$COMPREHENSIVE_REPORT"
    echo " Mail Servers: $mx_count" | tee -a "$COMPREHENSIVE_REPORT"
    echo " Name Servers: $ns_count" | tee -a "$COMPREHENSIVE_REPORT"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"

    echo " Security Observations:" | tee -a "$COMPREHENSIVE_REPORT"
    # Add automated security observations based on findings
    echo " - Check for SPF/DKIM/DMARC records in TXT" | tee -a
"$COMPREHENSIVE_REPORT"
    echo " - Verify name server security" | tee -a "$COMPREHENSIVE_REPORT"
}

```

```

    echo " - Analyze subdomain structure" | tee -a
"$COMPREHENSIVE_REPORT"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"
}

# Main execution
main() {
    if [ -z "$DOMAIN" ]; then
        echo "Usage: $0 <domain>"
        exit 1
    fi

    echo "🚀 Starting comprehensive reconnaissance..." | tee -a
"$COMPREHENSIVE_REPORT"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"

    advanced_ping_analysis
    comprehensive_dns_scan
    generate_summary

    echo "✅ Reconnaissance complete!" | tee -a
"$COMPREHENSIVE_REPORT"
    echo "📁 Results saved in: $OUTPUT_DIR" | tee -a
"$COMPREHENSIVE_REPORT"
    echo "📄 Main report: $COMPREHENSIVE_REPORT" | tee -a
"$COMPREHENSIVE_REPORT"
}

main "$@"
...

```

Step 2: Run Comprehensive Reconnaissance

bash

chmod +x advanced_recon.sh

./advanced_recon.sh example.com

```
pe here to sea...
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken W

kali-linux-2025.4-vmware-a...
1 2 3 4 5

Session Actions Edit View Help
GNU nano 9.0
#!/bin/bash
# Advanced Reconnaissance Framework
# Usage: ./advanced_recon.sh <domain>

DOMAIN="$1"
OUTPUT_DIR="recon_${DOMAIN}_${date +%Y%m%d_%H%M%S}"
COMPREHENSIVE_REPORT="${OUTPUT_DIR}/comprehensive_report.txt"

# Create output directory
mkdir -p "$OUTPUT_DIR"

echo "🔍 Advanced Reconnaissance Framework" | tee "$COMPREHENSIVE_REPORT"
echo "Target: $DOMAIN" | tee -a "$COMPREHENSIVE_REPORT"
echo "Date: $(date)" | tee -a "$COMPREHENSIVE_REPORT"
echo "===== " | tee -a "$COMPREHENSIVE_REPORT"

# Function to perform comprehensive ping analysis
advanced_ping_analysis() {
    echo "📡 Advanced Ping Analysis" | tee -a "$COMPREHENSIVE_REPORT"
    echo "===== " | tee -a "$COMPREHENSIVE_REPORT"

    local ping_file="${OUTPUT_DIR}/ping_analysis.txt"
    ping -c 10 "$DOMAIN" > "$ping_file"

    # Analyze ping statistics
    local avg_ping=$(grep "avg" "$ping_file" | awk -F '/' '{print $5}')
    local packet_loss=$(grep "packet loss" "$ping_file" | awk '{print $6}')

    echo "📊 Network Statistics:" | tee -a "$COMPREHENSIVE_REPORT"
    echo "   Average RTT: $avg_ping ms" | tee -a "$COMPREHENSIVE_REPORT"
    echo "   Packet Loss: $packet_loss" | tee -a "$COMPREHENSIVE_REPORT"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"
}

# Function for comprehensive DNS reconnaissance
comprehensive_dns_scan() {
    echo "🌐 Comprehensive DNS Analysis" | tee -a "$COMPREHENSIVE_REPORT"
    echo "===== " | tee -a "$COMPREHENSIVE_REPORT"

    local dns_file="${OUTPUT_DIR}/dns_records.txt"

    # Get all DNS record types
    for record in A AAAA MX TXT NS SOA CNAME; do
        echo "📄 $record Records:" | tee -a "$COMPREHENSIVE_REPORT"
    done
}

input to this VM, move the mouse pointer inside or press Ctrl+G
```

```
kali@kali: ~  
$ ./advanced_recon.sh google.com  
Advanced Reconnaissance Framework  
Target: google.com  
Date: Thu May 21 10:09:59 AM EDT 2026  
Starting comprehensive reconnaissance ...  
Advanced Ping Analysis  
Network Statistics:  
Average RTT: 236.148 ms  
Packet Loss: 0%  
Comprehensive DNS Analysis  
A Records:  
142.251.216.110  
AAAA Records:  
2c0f:fb50:4003:803::200e  
MX Records:  
10 smtp.google.com.  
TXT Records:  
;; communications error to 10.101.8.149#53: timed out  
"onetrust-domain-verification=0d477fe608074e6f9c12bca7826035cc"  
"onetrust-domain-verification=6d685f1d41a94696ad7ef771f68993e0"  
"google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4o8-0Hf5o"  
"docuSign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e"  
"cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"  
"apple-domain-verification=30afIBcvSuDV2PLX"  
"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"  
"MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB"  
"globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="  
"v=spf1 include:_spf.google.com ~all"  
"google-site-verification=4ibFUGB-wXLQ_S7vsXVomSTVamu0XBivAZpR5IZ87D0"  
"docuSign=1b0a6754-49b1-4db5-8540-d2c12664b289"  
"google-site-verification=TV9-DBe4R80X4v0M4U_bd_J9cp0JM0nikft0jAgjmsQ"  
NS Records:  
ns1.google.com.  
ns2.google.com.  
ns3.google.com.  
ns4.google.com.
```

The scan showed stable network performance with 0% packet loss and an average RTT of 236 ms. DNS analysis revealed IPv4 and IPv6 addresses, one mail server (smtp.google.com), and four authoritative name servers, indicating a highly redundant infrastructure.

TXT records included multiple domain verification entries and SPF configuration, showing strong email and domain security controls. Overall, the target demonstrates a well-structured and highly secured DNS and network architecture.

```
to sea...
puter
linux-2025.4-
sploitable2-
SP Broken V

kali-linux-2025.4-vmware-a...
1 2 3 4 5

Session Actions Edit View Help

(kali@kali)-[~]
$ ./basic_recon.sh google.com
Starting Basic Reconnaissance for: google.com

Ping Analysis for google.com

Target is reachable
PING google.com (172.217.22.46) 56(84) bytes of data.
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=208 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=146 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=291 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=210 ms

— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3010ms
rtt min/avg/max/mdev = 145.790/213.765/291.171/51.624 ms
IP Address: 172.217.22.46

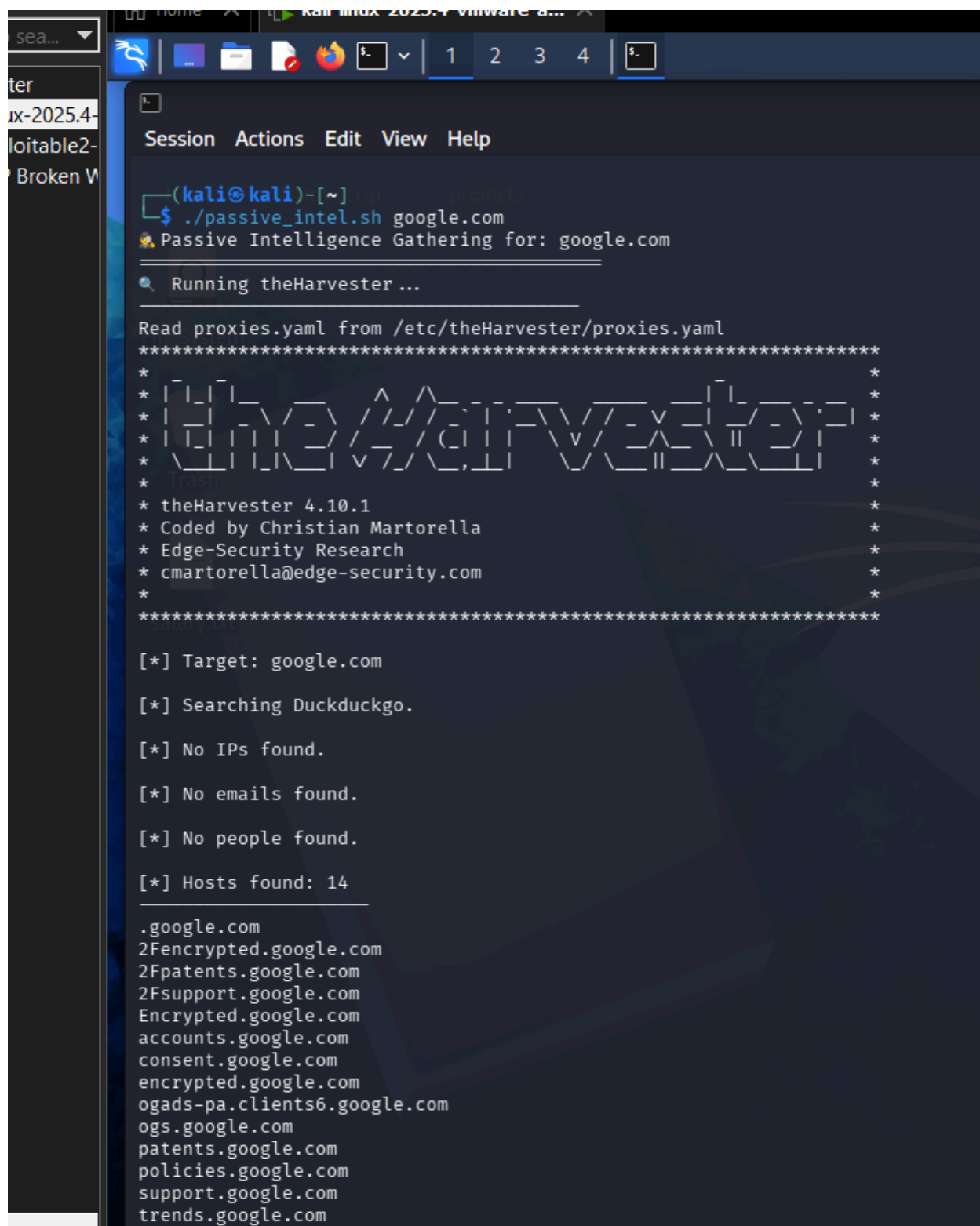
WHOIS Analysis for google.com

Registrar Information:
getaddrinfo(whois.verisign-grs.com): Temporary failure in name resolution

Dates Information:
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
NOTICE: The expiration date displayed in this record is the date the
currently set to expire. This date does not necessarily reflect the expiration
view the registrar's reported date of expiration for this registration.

Contact Information:
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrant Country: US
Registrant Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
Tech Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
contact, and such email address is not visible above, you may do so via our web

Name Servers:
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
Name Server: ns2.google.com
Name Server: ns3.google.com
Name Server: ns1.google.com
Name Server: ns4.google.com
```



```
(kali@kali)-[~]  
$ ./advanced_recon.sh google.com  
🔍 Advanced Reconnaissance Framework  
Target: google.com  
Date: Thu May 21 01:56:24 PM EDT 2026
```

🚀 Starting comprehensive reconnaissance ...

🔍 Advanced Ping Analysis

📊 Network Statistics:
Average RTT: 142.243 ms
Packet Loss: 10%

🌐 Comprehensive DNS Analysis

📄 A Records:
142.251.216.110

📄 AAAA Records:
2c0f:fb50:4003:803::200e

📄 MX Records:
;; communications error to 10.101.8.149#53: timed out
10 smtp.google.com.

📄 TXT Records:
;; communications error to 10.101.8.149#53: timed out
"google-site-verification=4ibFUGB-wXLQ_S7vsXVomSTVamuOXBivAzpR5IZ87D0"
"apple-domain-verification=30afIBcvSuDV2PLX"
"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"
"onetrust-domain-verification=0d477fe608074e6f9c12bca7826035cc"
"cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"
"v=spf1 include:_spf.google.com ~all"
"google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4oB-0Hf5o"
"docusign=1b0a6754-49b1-4db5-8540-d2c12664b289"
"MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB"
"google-site-verification=TV9-DBe4R80X4v0M4U_bd_J9cpOJM0nikft0jAgjmsQ"
"globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="
"docusign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e"
"onetrust-domain-verification=6d685f1d41a94696ad7ef771f68993e0"

📄 NS Records:
ns1.google.com.
ns2.google.com.
ns3.google.com.
ns4.google.com.

Critical Thinking Questions:

1. Legal and Ethical Considerations:

- What constitutes authorized vs unauthorized reconnaissance?

Authorized reconnaissance is conducted with proper permission from the organization or within an approved scope such as a penetration testing contract or lab environment. Unauthorized reconnaissance involves scanning or gathering information from systems without consent, which may violate laws and organizational policies.

- How do you ensure your activities remain within legal boundaries?

To remain within legal limits, reconnaissance activities should only target approved domains and systems. Security professionals must follow defined scopes, avoid disruptive actions, document permissions, and comply with cybersecurity laws and ethical guidelines.

2. Technical Analysis:

- What DNS misconfigurations commonly lead to security?

vulnerabilities? Common DNS misconfigurations include exposed zone transfers, missing SPF/DKIM/DMARC records, improperly configured name servers, and outdated DNS entries. These weaknesses can lead to email spoofing, information leakage, or unauthorized access.

- How can TTL values be used in network mapping?

TTL (Time To Live) values help estimate the number of network hops between a source and target system. Different operating systems also use different default

TTL values, allowing analysts to infer operating systems and understand network structure during reconnaissance.

3. Strategic Thinking:

- How would you prioritize findings for a penetration test?

High-risk findings such as exposed administrative portals, vulnerable services, weak email protections, and sensitive subdomains should be prioritized first because they may provide direct attack paths into the target environment.

- What additional reconnaissance would you conduct for a comprehensive assessment?

Additional reconnaissance may include port scanning, service enumeration, vulnerability scanning, and web application mapping to achieve a more complete security assessment.

ng, SSL certificate analysis, technology fingerprinting, employee OSINT gather

Recommendations

1. Regularly audit and remove unused or outdated subdomains to reduce attack surface.
2. Enforce full email security policies including SPF, DKIM, and DMARC.
3. Monitor DNS records and certificate transparency logs for unauthorized changes.
4. Implement continuous security monitoring for exposed services.
5. Restrict unnecessary public exposure of internal systems where possible.

Conclusion

The reconnaissance process successfully demonstrated how attackers or security analysts can gather critical information without direct access to systems. Domains analyzed showed strong infrastructure security, but also a large attack surface due to numerous subdomains and exposed public services.

Tools like `dig`, `nslookup`, and `theHarvester` proved essential for mapping DNS records, identifying mail servers, and discovering subdomains.